ДНІПРОВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ОЛЕСЯ ГОНЧАРА

Факультет прикладної математики та інформаційних технологій
Кафедра інженерії програмного забезпечення та інформаційних технологій

КУРСОВА РОБОТА

з дисципліни «Бази даних»

на тему: «Розроблення та тестування застосунку з використанням бази даних для керуючої компанії для багатоквартирного будинку»

Виконавець
студент групи ПЗ-24у-1
спеціальність
121 Інженерія програмного забезпечення

Кондрачук Олександр Вадимович

Керівник_____

_____

Кількість балів _____

Національна шкала _____

Члени комісії : _____ Леонід МАЩЕНКО
(підпис)

_____ Марина ІВАНЧЕНКО
(підпис)

_____ Світлана СІРИК
(підпис)

Дніпро – 2026

ЗМІСТ

ПОСТАНОВКА ЗАДАЧІ 3
1 ПРОЄКТУВАННЯ СИСТЕМИ БАЗ ДАНИХ 4
1.1. Обстеження предметної області 4
1.2. Основні задачі, що треба розв'язати 6
1.3. Основні запити за рівнями доступу користувачів 7
1.4. Нормалізація відношень та її обгрунтування 16
1.5. Побудова граф-схеми інфологічної моделі та перевірка її коректності 18
2 РЕАЛІЗАЦІЯ СИСТЕМИ БАЗ ДАНИХ ТА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ 20
2.1. Обґрунтування вибору СУБД та середовища розробки 20
2.2. Структурна схема програми та її опис 20
2.3. Опис інтерфейсу користувачів та засобів його розробки 22
2.4. Призначення основних процедур та функцій 22
2.5. Реалізація складних запитів на вибірку та звітів для користувачів різних рівнів 23
3 БЕЗПЕКА ТА СЕРВІСНІ ФУНКЦІЇ СИСТЕМИ 27
3.1. Реєстрація та автентифікація користувачів з різним доступом 27
3.2. Архівування, відновлення та інші функції з сервісу та налаштування 28
3.3. Ведення журналу роботи користувачів системи та його обробка 31
4 ІНСТРУКЦІЇ ПО ВИКОРИСТАННЮ ТА СУПРОВОДЖЕННЮ КОРИСТУВАЧАМ ЗА РІВНЕМ ДОСТУПУ 34
4.1 Інструкції по використанню та супроводженню адміністратору системи 35
4.2 Інструкції по використанню та супроводженню мешканця 38
4.3 Інструкції по використанню та супроводженню працівника
5 АНАЛІЗ ТА ОБГРУНТУВАННЯ РЕЗУЛЬТАТІВ ТЕСТУВАННЯ ТА НАЛАГОДЖЕННЯ 41
5.1. Початкове введення даних 41
5.2. Базове функціонування 44
5.3. Максимальне навантаження 48
ВИСНОВКИ 50
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ 51
ДОДАТОК 1 52
ДОДАТОК 2 54

ПОСТАНОВКА ЗАДАЧІ

Розробити базу даних засобами обраної СУБД та програмний застосунок (або сайт), що забезпечують введення, перегляд, редагування даних та виконання усіх запитів користувачів у відповідності до виявлених завдань за інформаційними потоками у предметній області.

Загальні вимоги до бази даних та роботи:

1. Провести всеосяжне обстеження предметної області;
2. Виявити основні інформаційні потоки та завдання користувачів, які необхідно розв’язати;
3. Визначити вид, правила формування і типи основних даних для вхідних та вихідних документів;
4. Виявити аномалії при роботі з об’єктами предметної області при виконанні основних операцій над даними та провести провести нормалізацію відношень;
5. Виконати побудову інфологічної моделі предметної області та перевірити її коректність;
6. Для відповідної інфологічної моделі виконати реалізацію в одній з реляційних СУБД;
7. Розробити індивідуальні форми (та сторінки сайту) для користувачів різних рівнів, що забезпечують дружній інтерфейс, орієнтований на предметну область. Ці форми мають надавати можливість навігації по таблиці та вирішення задач користувачів.
8. Врахувати можливість архівації та відновлення таблиць бази даних або всієї бази даних; створення копії бази даних для перенесення на інше робоче місце; можливість перегляду адміністратором журналу дій користувачів за відповідними запитами (фільтрами).
9. Створити запити різноманітних типів та звіти для основних запитів. Завдяки запитам на вибірку різної складності забезпечити надання необхідної інформації користувачам різних рівнів
10. Забезпечити контроль даних при введенні інформації по типу, діапазону, значенню та цілісність даних;
11. Виконати тестування та налагодження програмного продукту;
12. Розробити інструкції для користувачів різних рівнів;
13. Підготувати звіт з Курсової роботи.

1 ПРОЄКТУВАННЯ СИСТЕМИ БАЗ ДАНИХ

1.1. Обстеження предметної області

У якості предметної області було обрано “Керуюча компанія для багатоквартирного будинку. Розроблене рішення орієнтоване на автоматизацію ключових процесів: облік мешканців, квартир і будинків, нарахування платежів за послуги, фіксація оплат, обробка заявок на ремонт, а також внутрішня аналітика і контроль витрат.

Система дозволяє адміністраторам та працівникам компанії продивлятися мешканців та керувати їх записами, керувати інформацією про будинки, квартири, платежі, витрати, оголошення. Адміни також мають доступ до журналу подій, мають можливість створювати резерні копії бази даних, робити різноманітні звіти. У ході аналізу виникла потреба розробити інформаційну систему, яка дозволить налагодити прозору та ефективну комунікацію між мешканцями й керуючою компанією. Головна мета — автоматизувати повсякденні процеси у сфері ЖКГ, забезпечити своєчасний облік платежів і систематизувати дані про всі житлові об'єкти.

Було зроблено сутності:

1. **Користувач (users**). Всі, хто має доступ до системи – машканці, адміністратори та технічні працівники.
   - Username –ім’я користувача
   - password_hash – пароль
   - email – пошта
   - role – роль користувача: resident / admin / worker
   - created_at – дата реєстрації

2. **Мешканець (residents).** Зберігається інформація про мешканців.
   - full_name – ФІО
   - phone – телефон
   - apartment_id – зв’язок з житлом
   - building_id – зв’язок з домом
   - registred_at – дата реєстрації

3. **Квартира (apartments).** Інформація про квартиру.
   - building_id – зв’язок з будівлею
   - number – номер
   - floor – поверх
   - area - площа
   - owned – статус, чи зайнята
   - user_id - зв’язок з мешканцем

4. **Будівля (buildings).** Інформація про будівлі.
   - address – адреса
   - built_year – рік побудови
   - number_of_floores – кількість поверхів
5. **Платежі (payments).** Інформація про платежі.
   - resident_id – зв’язок з мешканцем
   - month – місяць
   - amount – сума
   - paid_at – дата оплати
   - status - статус
6. **Запити мешканців (maintenance_requests).** Інформація про потреби мешканців.
   - resident_id – зв’язок з мешканцем
   - apartment_id - зв’язок з квартирою
   - description - опис
   - status - статус
7. **Замовлення для працівників (work_orders).** Інформація щодо замовлень для працівників.
   - request_id – зв’язок з тим, хто зробив запит
   - worker_id – зв’язок з працівником, який працює над замовленням
   - scheduled_date – запланована дата
   - completed_at – дата закінчення
   - notes – нотатки
8. **Витрати(expenses).** Інформація про витрати.
   - description - опис
   - amount - сума
   - date - дата
   - category – категорія
9. **Оголошення(announcements).** Інформація про оголошення.
   - title - заголовок
   - message - повідомлення
   - author_id – автор оголошення
10. **Журнал подій (event_logs).** Інформація про подію.
    - user_id - зв’язок з автором події
    - event_type – тип події
    - event_time – час події
    - details – деталі

11. **Резервні копії(backup_logs)**. Інформація про резервні копії.
    - backup_type – тип копії
    - backup_date – дата копії
    - perfomed_by – ким зроблена
    - notes - повідомлення

1.2. Основні задачі, що треба розв’язати

Для ефективного управління багатоквартирним будинком наша система реалізує наступні основні функції, доступні для різних категорій користувачів:

1. **Авторизація користувачів:**

Кожен мешканець, адміністратор або працівник може увійти в систему, використовуючи свій логін і пароль.
Категорії користувачів: Мешканець, Адміністратор, Працівник.
Доступ: реєстрація здійснюється особисто користувачем. В залежності від ролі доступні різні форми, для контроля доступу за ролями використовується sessionStorage для зберігання ролі поточного юзера.

2. **Керування мешканцями:**

Адміністратори можуть додавати інформацію про нових мешканців, переглядати їхні дані, редагувати або видаляти їх за потреби.
Категорії користувачів: Адміністратор.
Доступ: Тільки для авторизованих адміністраторів.

3. **Керування квартирами:**

Адміністратори мають можливість додавати інформацію про нові квартири, вказуючи їх номер, поверх, площу та власника. Також можна переглядати, редагувати або видаляти дані про існуючі квартири.
Категорії користувачів: Адміністратор.
Доступ: Тільки для авторизованих адміністраторів.

4. **Керування будинками:**

Адміністратори можуть надавати інформацію про нові будинки, включаючи адресу, рік побудови та кількість поверхів. Доступний перегляд, редагування та видалення інформації про будинки.
Категорії користувачів: Адміністратор.
Вимоги до доступу: Тільки для авторизованих адміністраторів.

5. **Облік платежів:**

Мешканці та адміністратори можуть переглядати інформацію про платежі. Адміністратори мають можливість фіксувати нові платежі, вказуючи період, суму та статус оплати.
Категорії користувачів: Мешканець, Адміністратор.
Вимоги до доступу: Для перегляду та оплати – авторизованим мешканцям та адміністраторам, для внесення – тільки адміністраторам.

6. **Обробка заявок на обслуговування:**

Мешканці можусть створювати заявки на ремонт або технічне обслуговування, описуючи проблему. Працівники можуть переглядати ці заявки та змінювати їх статус. Адміністратори мають повний доступ до всіх

заявок.
Категорії користувачів: Мешканець, Працівник, Адміністратор.
Вимоги до доступу: Створення – авторизованим мешканцям, перегляд та зміна статусу – авторизованим працівникам та адміністраторам.

7. Керування робочими нарядами:

Працівники можуть переглядати призначені їм робочі наряди, вказувати дату виконання та додавати примітки. Адміністратори можуть створювати нові наряди, призначати виконавців та контролювати їх виконання.
Категорії користувачів: Працівник, Адміністратор.
Вимоги до доступу: Перегляд – авторизованим працівникам та адміністраторам, створення та редагування – тільки адміністраторам.

8. Облік витрат:

Адміністратори та працівники можуть фіксувати витрати, вказуючи опис, суму, дату та категорію.
Категорії користувачів: Працівник, Адміністратор.
Вимоги до доступу: Тільки для авторизованих працівників та адміністраторів.

9. Оголошення:

Адміністратори можуть створювати та публікувати оголошення для мешканців. Мешканці та адміністратори можуть переглядати ці оголошення.
Категорії користувачів: Мешканець, Адміністратор.
Вимоги до доступу: Перегляд – для всіх авторизованих користувачів, створення – тільки адміністраторам.

10. Ведення журналу подій:

Система автоматично записує всі важливі дії користувачів. Адміністратори можуть переглядати цей журнал, фільтрувати та сортувати записи для контролю та аудиту.
Категорії користувачів: Адміністратор.
Доступ: Тільки для авторизованих адміністраторів.

11. Резервне копіювання та відновлення:

Адміністратори мають можливість створювати резервні копії бази даних, завантажувати їх та відновлювати систему з попередніх копій для забезпечення збереження даних.
Категорії користувачів: Адміністратор.
Доступ: Тільки для авторизованих адміністраторів.

12. **Звіти**

Адміністратори можуть генерувати різноманітні звіти на основі даних системи для аналізу та прийняття рішень
Категорії користувачів: Адміністратор.
Доступ: Тільки для авторизованих адміністраторів.

1.3. Основні запити за рівнями доступу користувачів

У системі управління багатоквартирним будинком реалізовано набір SQL-запитів, що відповідають ключовим операціям для різних сутностей та рівнів доступу користувачів.

Керування користувачами (Адміністратор):

Адміністратори мають можливість створювати, змінювати та видаляти облікові записи користувачів (мешканців, адміністраторів, працівників). **Приклади запитів:**

Створення нового користувача:

INSERT INTO users (username, password_hash, email, role) VALUES ('new_user', sdsadadada', 'user@example.com', 'resident');

Оновлення інформації про користувача:

UPDATE users SET email = 'updated@example.com', role = 'admin' WHERE id = 7;

Видалення користувача:

DELETE FROM users WHERE id = 7;

Звіт за кількістю ролей в таблиці користувачів:

SELECT role, COUNT(*) as count FROM users GROUP BY role ORDER By count DESC;

Керування мешканцями (Адміністратор):

Адміністратори відповідають за додавання, редагування та видалення інформації про мешканців будинку.

Приклади запитів:

Додавання нового мешканця:

INSERT INTO residents (user_id, full_name, apartment_id, building_id, phone, registered_at) VALUES (8, 'resident1', 101, 1, '+380671234567', '2025-05-16');

Редагування даних мешканця:

UPDATE residents SET phone = '+380509876543', apartment_id = 102 WHERE id = 3;

Видалення мешканця:

DELETE FROM residents WHERE id = 3;

Пошук клієнтів за поверхом у будівлі:

SELECT r.full_name, a.number AS apartment_number FROM residents r JOIN apartments a ON r.apartment_id = a.id WHERE a.building_id = 1 AND a.floor = 2;

Керування квартирами (Адміністратор):

Адміністратори керують інформацією про квартири в будинках.

Приклади запитів:

Додавання нової квартири:

INSERT INTO apartments (building_id, number, floor, area, owned, user_id) VALUES (1, '205', 2, 65.5, TRUE, 9);

Оновлення даних квартири:
UPDATE apartments SET area = 70.0, owned = TRUE WHERE id = 15;

Видалення квартири:
DELETE FROM apartments WHERE id = 15;

Пошук інформації про квартиру у будівлі:
SELECT b.address, COUNT(a.id) AS apartment_count FROM buildings b JOIN apartments a ON b.id = a.building_id GROUP BY b.address ORDER BY apartment_count DESC;

Облік платежів (Мешканець, Адміністратор):

Мешканці можуть переглядати свої платежі, адміністратори – додавати, редагувати та переглядати всі платежі. Приклади запитів:

Додавання нового платежу (Адміністратор):
INSERT INTO payments (resident_id, month, amount, paid_at, status) VALUES (8, '2025-06-01', 150.00, CURRENT_TIMESTAMP, 'payed');

Редагування платежу (Адміністратор):
UPDATE payments SET amount = 160.00, status = 'not payed' WHERE id = 25;

Перегляд платежів мешканця (Мешканець/Адміністратор):
SELECT id, month, amount, paid_at, status FROM payments WHERE resident_id = 8 ORDER BY month;

Заявки на обслуговування (Мешканець, Працівник, Адміністратор):

Мешканці створюють заявки, працівники оновлюють їх статус, адміністратори мають повний доступ. Приклади запитів:

Створення нової заявки (Мешканець):
INSERT INTO maintenance_requests (resident_id, apartment_id, description) VALUES (8, 101, 'Протікає кран на кухні.');

Оновлення статусу заявки (Працівник/Адміністратор):
UPDATE maintenance_requests SET status = 'in work', updated_at = CURRENT_TIMESTAMP WHERE id = 12;

Перегляд заявок мешканця (Мешканець/Адміністратор):
SELECT id, description, status, created_at FROM maintenance_requests WHERE resident_id = 8 ORDER BY created_at DESC;

Журнал подій (Адміністратор):

Адміністратори мають доступ до журналу дій користувачів. Приклади запитів:

Перегляд усіх подій:
SELECT el.event_time, u.username, el.event_type, el.details FROM event_log el LEFT JOIN users u ON el.user_id = u.id ORDER BY el.event_time DESC;

Фільтрація подій за типом або користувачем:

SELECT el.event_time, u.username, el.details FROM event_log el LEFT JOIN users u ON el.user_id = u.id WHERE el.event_type = 'створення' AND u.username = 'admin' ORDER BY el.event_time DESC;

Резервне копіювання (Адміністратор):

Створення запису про резервну копію:

```
INSERT INTO backup_logs (backup_type, performed_by, notes) VALUES
('copy', 1, 'Щоденна копія бази даних.');
```

Перегляд історії резервних копій:

```
SELECT backup_date, backup_type, notes, (SELECT username FROM users
WHERE id = backup_logs.performed_by) AS performed_by FROM backup_logs
ORDER BY backup_date DESC;
```

Ці SQL-запити є основою для функціонування системи та забезпечують виконання ключових завдань відповідно до ролей користувачів. Доступ до цих запитів реалізовано через API-ендпоінти з відповідною авторизацією для забезпечення безпеки та контролю даних.

1.4. Нормалізація відношень та її обгрунтування

Перша нормальна форма (1НФ)

На першому етапі проєктування бази даних було виконано приведення всіх відношень до першої нормальної форми (1НФ). Основною вимогою цієї форми є атомарність даних, тобто кожен атрибут повинен містити лише одне значення, а в таблицях не повинно бути повторюваних груп або множин значень в одному полі.

У межах 1НФ для кожного з дванадцяти відношень було визначено унікальний первинний ключ (PK), який забезпечує однозначну ідентифікацію кожного запису. Під час нормалізації повторювані або логічно незалежні групи даних були винесені в окремі сутності. Наприклад, інформація про заявки на ремонт не зберігається безпосередньо у таблиці квартир або мешканців, а винесена у самостійне відношення maintenance_requests. Кожен запис цієї таблиці містить атомарні дані про тип ремонту, статус виконання, дату створення та опис проблеми, що значно спрощує облік і контроль технічного обслуговування житлового комплексу.

У результаті було сформовано перелік сутностей бази даних:

– users — дані користувачів системи, їх ролі та параметри авторизації.

– buildings — інформація про житлові будинки та їх характеристики.

– apartments — дані про квартири, їх площу, поверх та власника.

– residents — персональна та контактна інформація мешканців.

– payments — облік оплат комунальних або сервісних послуг.

– services — перелік доступних послуг та їх вартість.

– maintenance_requests — заявки мешканців на ремонтні роботи.

– work_orders — інформація про виконання ремонтних робіт працівниками.

– expenses — фінансові витрати та категорії витрат системи.

– announcements — системні оголошення для мешканців.

– event_log — журнал системних подій та дій користувачів.

– backup_logs — журнал створення резервних копій та відновлення системи.

Друга нормальна форма (2НФ)

Процес приведення бази даних до другої нормальної форми (2НФ) передбачав перевірку всіх відношень, що вже перебувають у 1НФ, на відсутність часткових залежностей. Згідно з вимогами 2НФ, кожен неключовий атрибут повинен повністю функціонально залежати від усього первинного ключа, а не лише від його частини.

Під час аналізу структури інформаційної системи було встановлено, що всі таблиці використовують прості первинні ключі, які складаються лише з одного атрибута (id, backup_id тощо). Через це виникнення часткових залежностей є неможливим. Кожен неключовий атрибут повністю визначається відповідним первинним ключем.

Наприклад:

– у таблиці buildings усі характеристики будинку (address, built_year, number_of_floors) залежать лише від id;

– у таблиці apartments параметри квартири (number, floor, area, owned) визначаються тільки ключем id;

– у таблиці payments дані про суму платежу, дату оплати та статус залежать виключно від id платежу;

– у таблиці maintenance_requests опис ремонту, його тип та статус функціонально залежать лише від ідентифікатора заявки.

Таким чином, усі відношення розробленої бази даних автоматично задовольняють вимоги другої нормальної форми. Це забезпечує цілісність структури та виключає аномалії, пов’язані з частковими залежностями.

Третя нормальна форма (3НФ)

На фінальному етапі нормалізації було виконано перевірку відношень на відповідність третій нормальній формі. Основною вимогою 3НФ є відсутність транзитивних залежностей, тобто ситуацій, коли неключовий атрибут залежить від первинного ключа через інший неключовий атрибут.

У межах системи ця вимога реалізована шляхом розділення логічно незалежних сутностей. Наприклад, інформація про будинки зберігається окремо у таблиці buildings, тоді як таблиця apartments містить лише зовнішній ключ building_id. Це дозволяє уникнути дублювання адреси та характеристик будинку для кожної квартири.

Аналогічно, персональні дані мешканців винесені у таблицю residents, а інформація про користувачів системи авторизації — у таблицю users. Завдяки цьому зміна контактних або облікових даних користувача не потребує модифікації інших таблиць і не призводить до дублювання інформації.

У таблиці maintenance_requests зберігаються лише дані, що безпосередньо стосуються заявки на ремонт, тоді як інформація про виконавця робіт винесена у таблицю work_orders. Це дозволило уникнути транзитивної залежності між заявкою та працівником через дані виконання робіт.

Також журнал подій (event_log) та журнал резервного копіювання (backup_logs) реалізовані як окремі сутності, оскільки їх атрибути описують незалежні системні процеси та не повинні дублюватися в інших таблицях.

Таким чином, усі дванадцять відношень інформаційної системи приведено до третьої нормальної форми. Це забезпечує мінімізацію надлишковості даних, структурну цілісність бази даних та захист від аномалій вставки, оновлення і видалення.

1.5. Побудова граф-схеми інфологічної моделі та перевірка її коректності

Оскільки концептуальна модель бази даних із самого початку проектувалася відповідно до принципів реляційної нормалізації, більшість потенційних багатозначних та транзитивних залежностей були винесені в окремі сутності ще на етапі розробки структури. Після перевірки на відповідність третій нормальній формі кількість відношень не змінилася та становить 12 оптимізованих таблиць.

У процесі нормалізації не виникла потреба у додатковому розщепленні чи об’єднанні відношень, тому логічна структура бази даних залишилася незмінною. Для підтвердження правильності побудови інфологічної моделі було проведено перевірку граф-схеми за основними критеріями коректності.

Зв'язність і повнота. У процесі аналізу встановлено, що граф-схема є повністю зв'язною. Усі сутності бази даних пов'язані між собою через логічні або функціональні залежності, реалізовані за допомогою зовнішніх ключів. Ізольовані вершини, які не мають зв'язків з іншими елементами моделі, відсутні. Кожен зв'язок між сутностями має логічне завершення та відповідає вимогам предметної області. Таким чином, усі гілки граф-схеми є завершеними та беруть участь у загальній структурі інформаційної системи.

Початок і кінець. Граф-схема має єдину точку входу, яка відповідає початковому формуванню та зберіганню даних про користувачів, будинки та квартири. Подальша обробка інформації відбувається через пов'язані сутності системи. Кінцевими вершинами виступають таблиці, що зберігають результати діяльності системи або службову інформацію, зокрема Payments, WorkOrders, Announcements, EventLog та BackupLogs. Таким чином, структура моделі містить один вхід і декілька логічно визначених виходів.

Відсутність безвихідних зон. Перевірка показала, що кожна сутність граф-схеми, крім кінцевих, має хоча б один вихідний зв'язок до інших елементів моделі. Наприклад, сутність Buildings пов'язана з Apartments, Apartments — з Residents, а Residents — з Payments та MaintenanceRequests. Усі маршрути проходження даних приводять до кінцевих сутностей або завершуються збереженням результатів у відповідних таблицях. Безвихідні зони та цикли, що не мають логічного завершення, відсутні.

Коректність розгалужень. Для всіх логічних зв'язків між сутностями передбачено обробку всіх можливих варіантів розвитку подій. Наприклад, користувач може мати одну з ролей (resident, admin або worker), заявка на ремонт може перебувати у станах new, in work або completed, а платіж може

мати статус payed або not payed. Для кожного можливого значення в структурі бази даних передбачено відповідні атрибути, обмеження та зв'язки, що забезпечують повноту опису предметної області та виключають неоднозначність обробки даних.

Отже, граф-схема інфологічної моделі відповідає всім критеріям коректності: є зв'язною та повною, містить визначені точки початку і завершення, не має безвихідних зон та забезпечує коректну реалізацію всіх необхідних розгалужень. Це підтверджує правильність побудови моделі та її готовність до подальшої реалізації у вигляді реляційної бази даних.

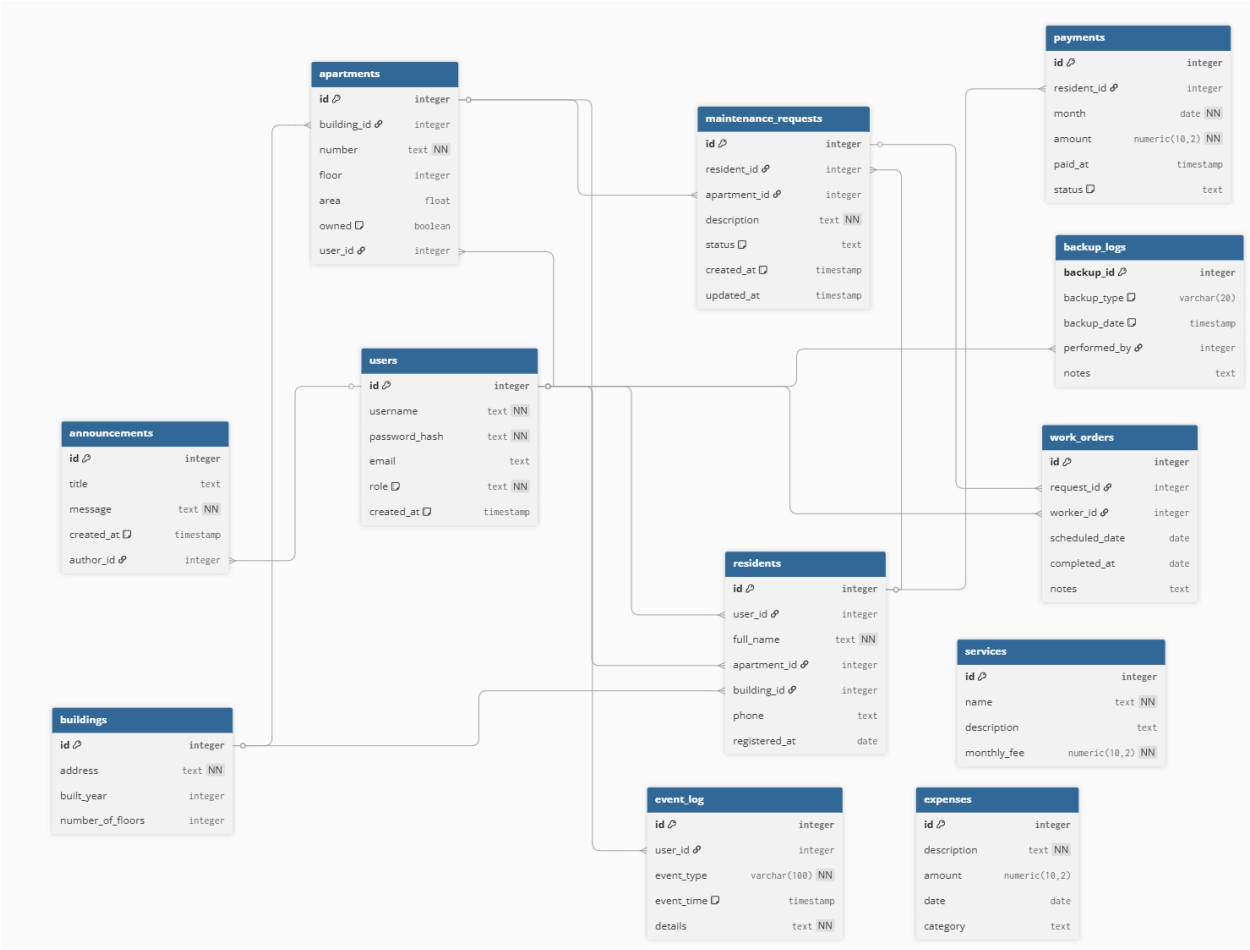


Рисунок 1.1 – ER-діаграма

2 РЕАЛІЗАЦІЯ СИСТЕМИ БАЗ ДАНИХ ТА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1. Обґрунтування вибору СУБД та середовища розробки

Для розробки бекенд-частини обрана платформа Node.js з фреймворком Express, що забезпечує швидкість, масштабованість і простоту побудови REST API.

Використання PostgreSQL як системи керування базами даних гарантує надійність збереження інформації, підтримку складних транзакцій і високий рівень цілісності даних.

Фронтенд реалізовано на React, що дозволяє створювати швидкий та інтерактивний інтерфейс користувача. Для зручного управління станом застосунку застосовується Redux Toolkit, який централізує логіку стану і спрощує масштабування. Комунікація між фронтендом і сервером здійснюється через REST API, що забезпечує чіткий розподіл обов'язків і легкість інтеграції.

Для роботи з базою даних використовується драйвер pg (node-postgres), який дозволяє виконувати прямі SQL-запити, забезпечуючи максимальний контроль над операціями і можливість тонкого налаштування продуктивності. Такий підхід не тільки підвищує продуктивність і надійність системи, а й спрощує її підтримку і розширення в майбутньому.

Вся архітектура орієнтована на чистоту коду, масштабованість і швидку реакцію на зміни бізнес-вимог.

2.2. Структурна схема програми та її опис

Система побудована на чіткому розподілі ролей і прав доступу. Вхід в неї починається з авторизації користувача та застосування sessionStorage збереження даних користувача, після чого відбувається маршрутизація до потрібних розділів інтерфейсу залежно від ролі.

Адміністратор отримує великий доступ: може керувати мешканцями, квартирами, будинками, контролювати платежі, переглядати журнал подій, працювати з резервними копіями і формувати звіти. Йому відкриті всі розділи системи, що забезпечує повний контроль над управлінням житловим комплексом.

Жителі (residents) мають обмежені права — можуть переглядати свої платежі, подавати заявки на обслуговування і читати оголошення. Вони не мають доступу до адміністративних функцій.

Працівники (worker) працюють із заявками на обслуговування і робочими нарядами, а також можуть фіксувати витрати, пов'язані з обслуговуванням.

Всі дії користувачів логуються в систему, що забезпечує прозорість і безпеку. Контроль доступу реалізований через перевірку ролей у sessionStorage, що запобігає несанкціонованому доступу.

2.3. Опис інтерфейсу користувачів та засобів його розробки

Фронтенд розроблено на React із чіткою компонентною структурою, що забезпечує легке масштабування та підтримку коду. Інтерфейс динамічно адаптується відповідно до ролі користувача, відображаючи лише релевантні розділи.

Адміністратор має доступ до дашборду зі статистикою, а також до сторінок для управління мешканцями, квартирами, будинками, платежами та звітами. Реалізовано функції експорту та друку PDF-звітів, а також перегляд журналу дій користувачів.

Жителі та працівники користуються спрощеним інтерфейсом: жителі можуть подавати заявки на обслуговування та контролювати свої платежі, працівники — управляти робочими завданнями і фіксувати витрати.

Така архітектура фронтенду забезпечує гнучкість та зручність розробки..

2.4. Призначення основних процедур та функцій

– **Авторизація**: Всі користувачі зберігаються в таблиці users. Тут є унікальний логін (username), захищений пароль (хеш), емейл і роль (resident, admin, worker). Для доступу до системи створюється JWT-токен, який підтверджує права користувача і визначає, що він може бачити і робити.

– **Журнал дій**: Таблиця event_log фіксує всі важливі події: створення, редагування, видалення записів. Тут зберігаються хто, коли і що зробив, а також деталі операції — це незамінний інструмент для відстеження і аудиту дій користувачів.

– **Жителі**: Таблиця residents пов’язує користувачів із конкретними квартирами (apartments) і будинками (buildings). Тут зберігається повне ім’я, телефон, дата реєстрації. Це допомагає ідентифікувати мешканців та управляти їхніми правами.

– **Квартири та будинки apartments** містить інформацію про номер, поверх, площу, власність і відповідального користувача. buildings зберігає адресу, рік побудови і кількість поверхів. Ці таблиці створюють базову структуру житлового фонду.

– **Платежі**: У таблиці payments зберігаються записи про платежі мешканців: місяць, сума, дата оплати і статус (payed або not payed). Система автоматично оновлює статуси і відстежує правильність сум.

– **Заявки на обслуговування і робочі наряди**: maintenance_requests — це звернення жителів із описом проблеми, статусом заявки і часовими мітками. work_orders пов’язує заявки з працівниками, планує дати робіт і фіксує завершення з нотатками.

– **Витрати**: Таблиця expenses служить для обліку витрат із категоріями, сумами та датами. Це допомагає контролювати бюджет і звітувати перед адміністрацією.

– **Оголошення**: В announcements зберігаються повідомлення для користувачів із заголовком, текстом, датою створення і автором. Це канал комунікації між адміністрацією і жителями.

– **Звіти**: На основі даних із вищенаведених таблиць формується звітність у форматі PDF з можливістю застосовувати різні фільтри — це спрощує аналіз і презентацію інформації.

– **Резервне копіювання**: Таблиця backup_logs відстежує створення і відновлення резервних копій бази даних із позначками часу, типом операції і користувачем, що її виконав. Це гарантія безпеки і збереження даних.

2.5. Реалізація складних запитів на вибірку та звітів для користувачів різних рівнів

Реалізація запитів до бази даних відбувається за допомогою нативних SQL-запитів через драйвер pg (PostgreSQL) у Node.js. Для складних вибірок пишу ручні запити з використанням JOIN, GROUP BY, WHERE та інших SQL-конструкцій.

Наприклад:

Звіт по ролях користувачів: SELECT payments.*, residents.full_name, apartments.number AS apartment_number FROM payments JOIN residents ON payments.resident_id = residents.id LEFT JOIN apartments ON residents.apartment_id = apartments.id WHERE residents.id = $1;

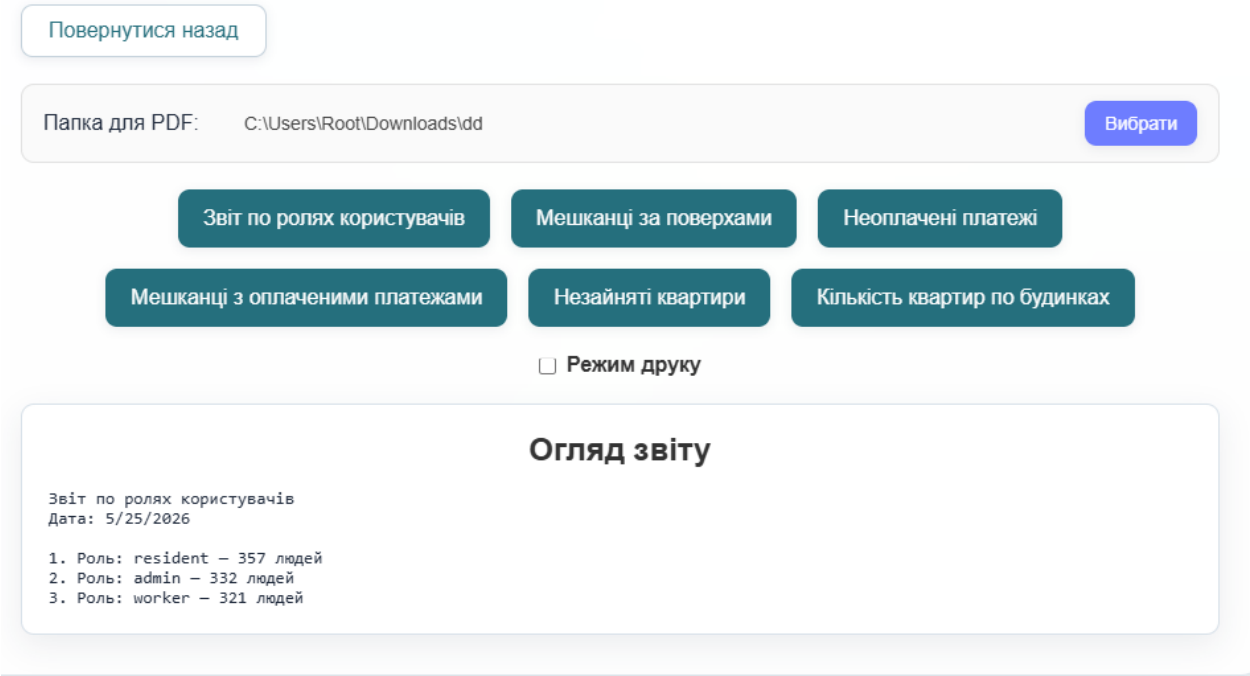


Рисунок 2.1 – Звіт по ролях користувачів

Мешканці за поверхами: SELECT * FROM event_log WHERE event_type = 'Updated' AND user_id = $1;

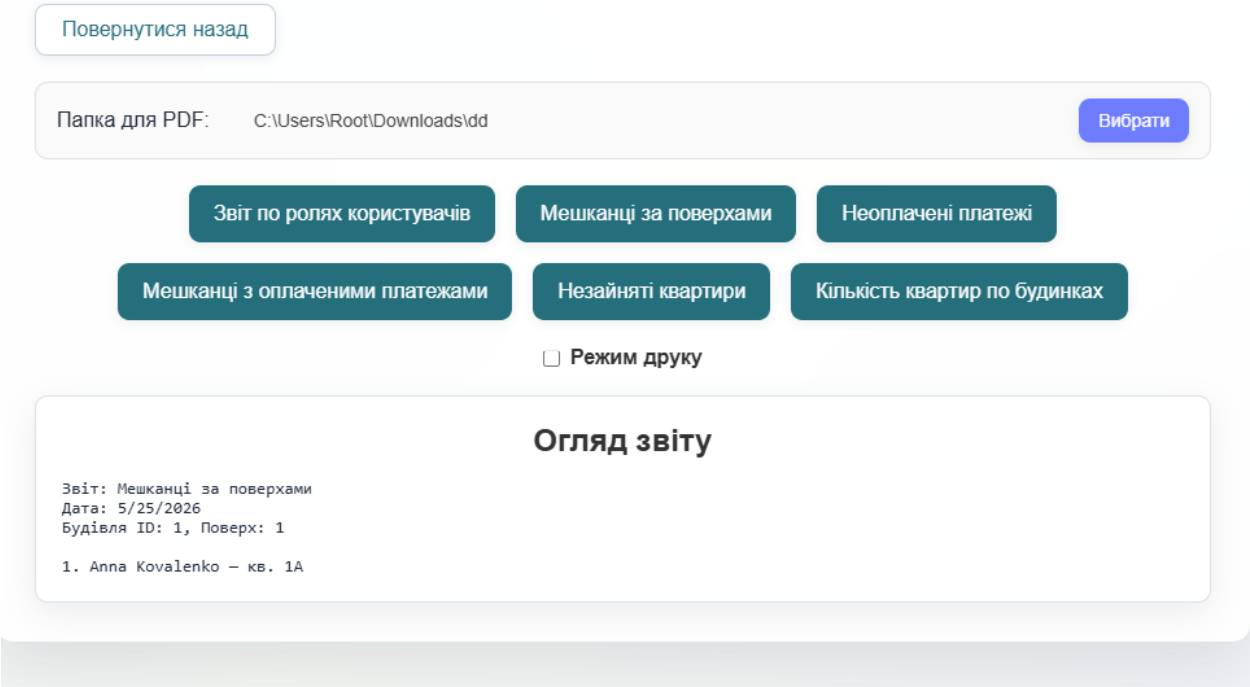


Рисунок 2.2 – Звіт по мешканцях за поверхами

Мешканці з оплаченими платежами: SELECT status, COUNT(*) AS count FROM payments GROUP BY status;

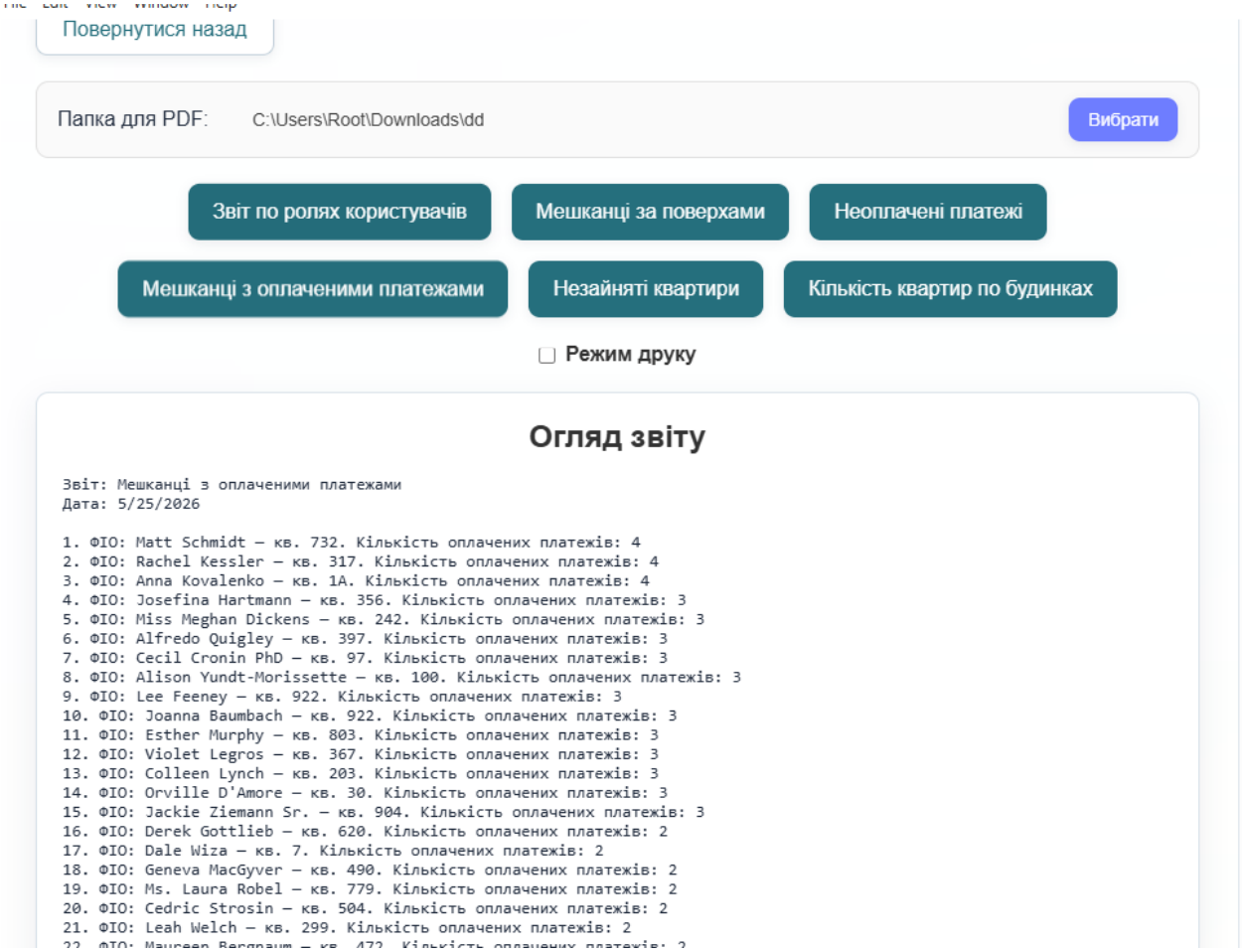


Рисунок 2.3 – Звіт по оплаченим платежам

Кількість квартир по будинках: SELECT apartments.*, buildings.id, apartments.number AS apartment_number FROM payments JOIN buildings ON buildings.building.id = residents.id LEFT JOIN apartments ON residents.apartment_id = apartments.id WHERE residents.id = $1;

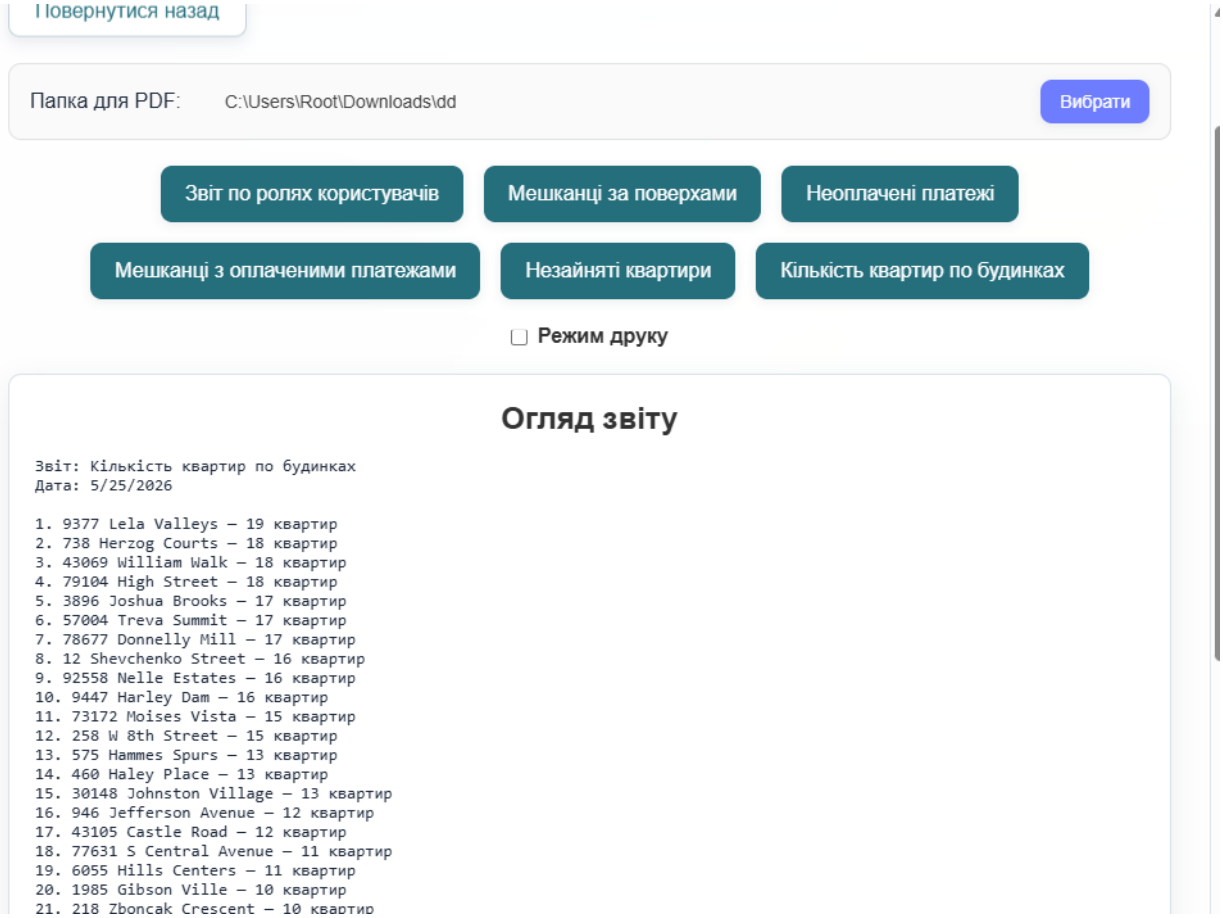


Рисунок 2.4 – Звіт по кількості квартир по будинках

Щоб створювати та друкувати PDF файли, я використовую JavaScript бібліотеку “jspdf”, що легко та швидко дозволяє формувати дані та відображати їх у PDF файлі. Завдяки такому підходу адміністратор може генерувати звіти за періодами, подіями користувачів та іншим таблицям. Завдяки всьому цьому, ми отримуємо швидкий, простий для роботи сервіс з різноманітними запитами та можливостями.

3 БЕЗПЕКА ТА СЕРВІСНІ ФУНКЦІЇ СИСТЕМИ

3.1. Реєстрація та автентифікація користувачів з різним доступом

Всього в нашій програмі передбачено 3 ролі користувачів: адміністратор, працівник, мешканець. При реєстрації у користувача є вибір обирати роль від свої потреб.

Для успішної реєстрації, в залежності від ролі, потрібно ввести різну кількість даних. Наприклад, мешканцю доведеться вводити повну інформацію про себе, квартиру та будинок, а адміністратору та працівнику буде достатньо лише юзернейму, паролю та пошти.

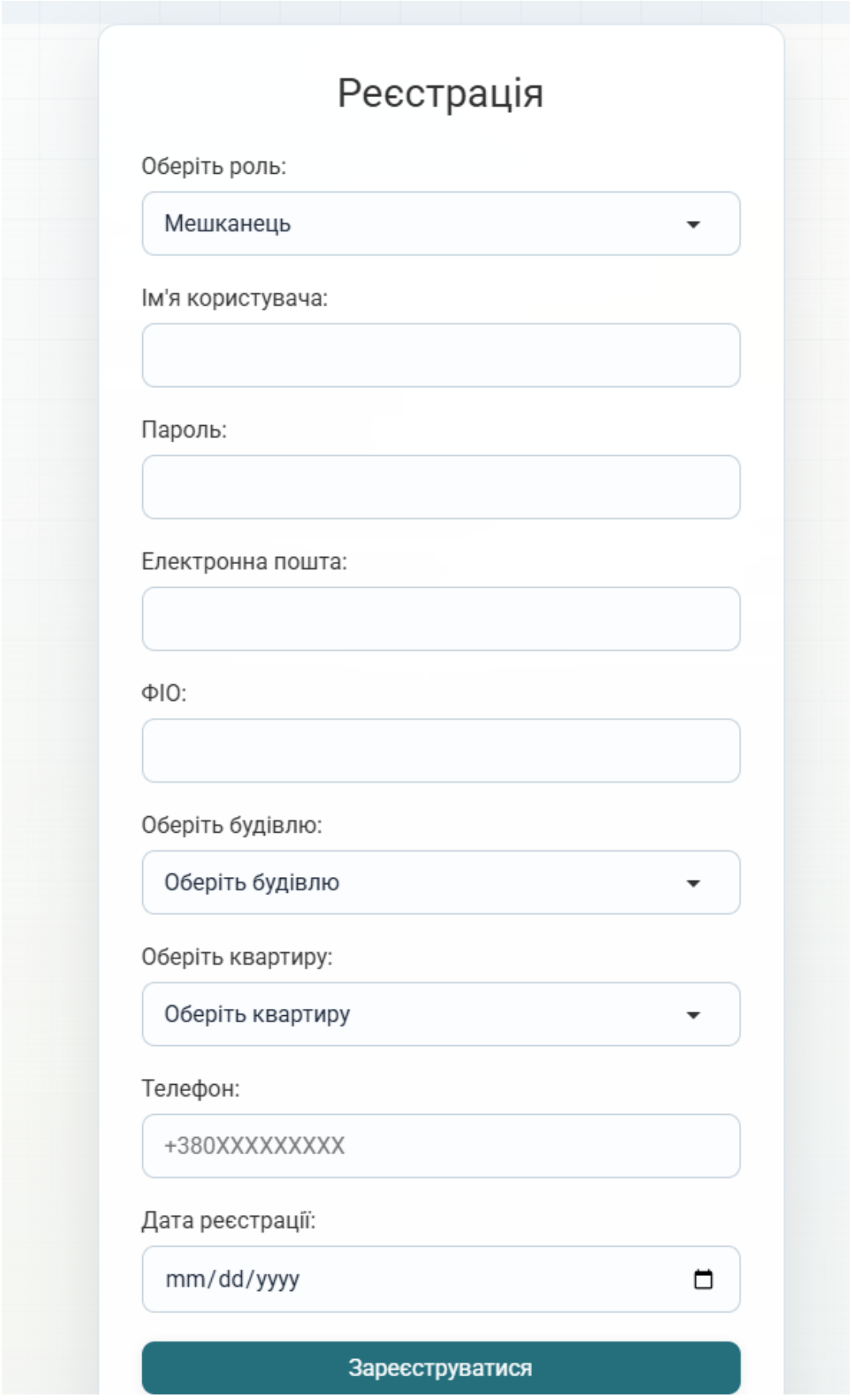


Рисунок 3.1 – Форма реєстрації мешканця

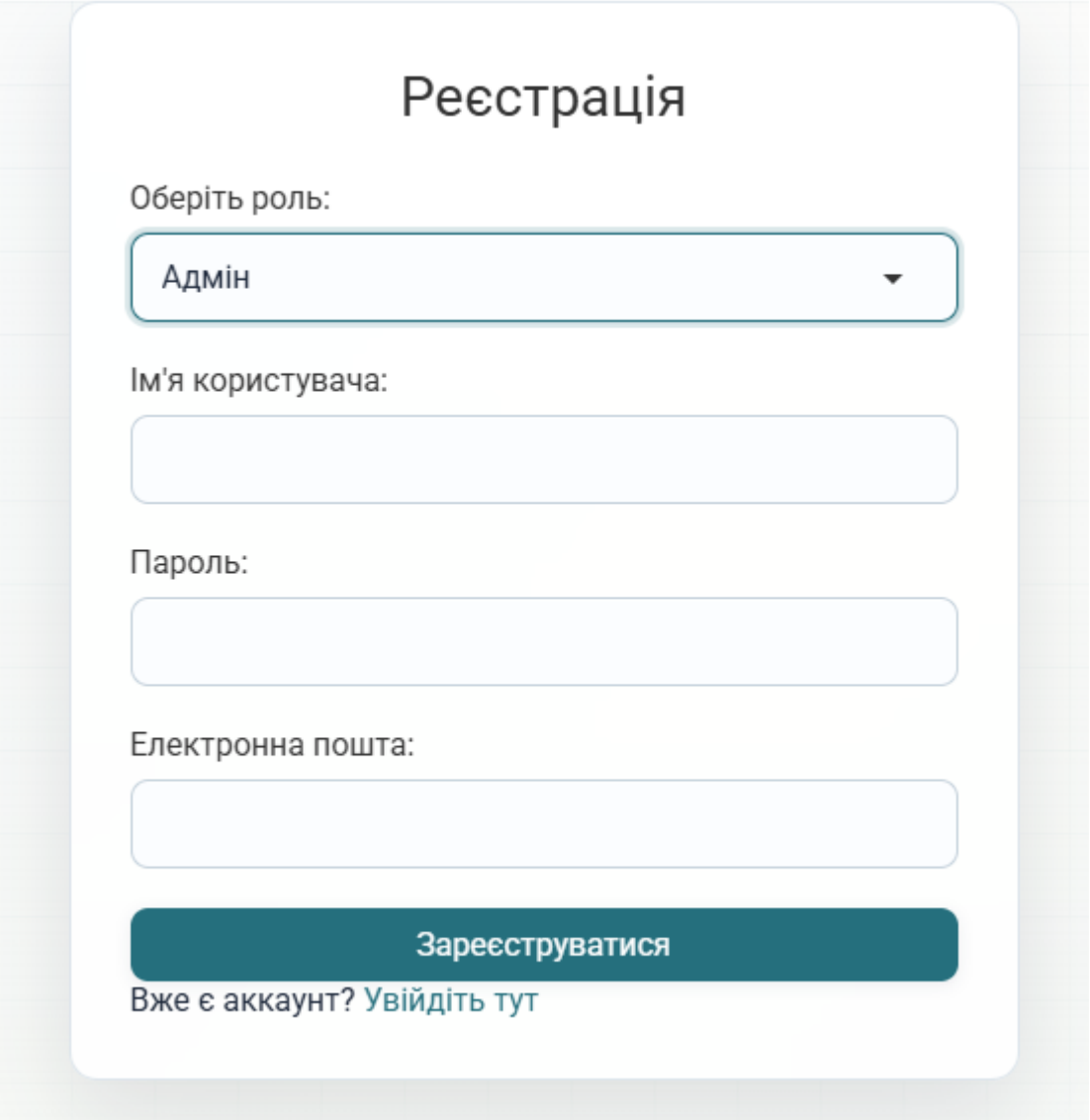


Рисунок 3.2 – Форма реєстрації адміністратора

Після успішної реєстрації, формується sessionStorage з даними, а головне – з ролью користувача. В майбутньому при перевірці доступу до сторінки буде використовуватися саме sessionStorage.

Також є можливість для всіх типів користувачів увійти до свого акаунта за допомогою сторінки логіну(див. рисунок 3.3)

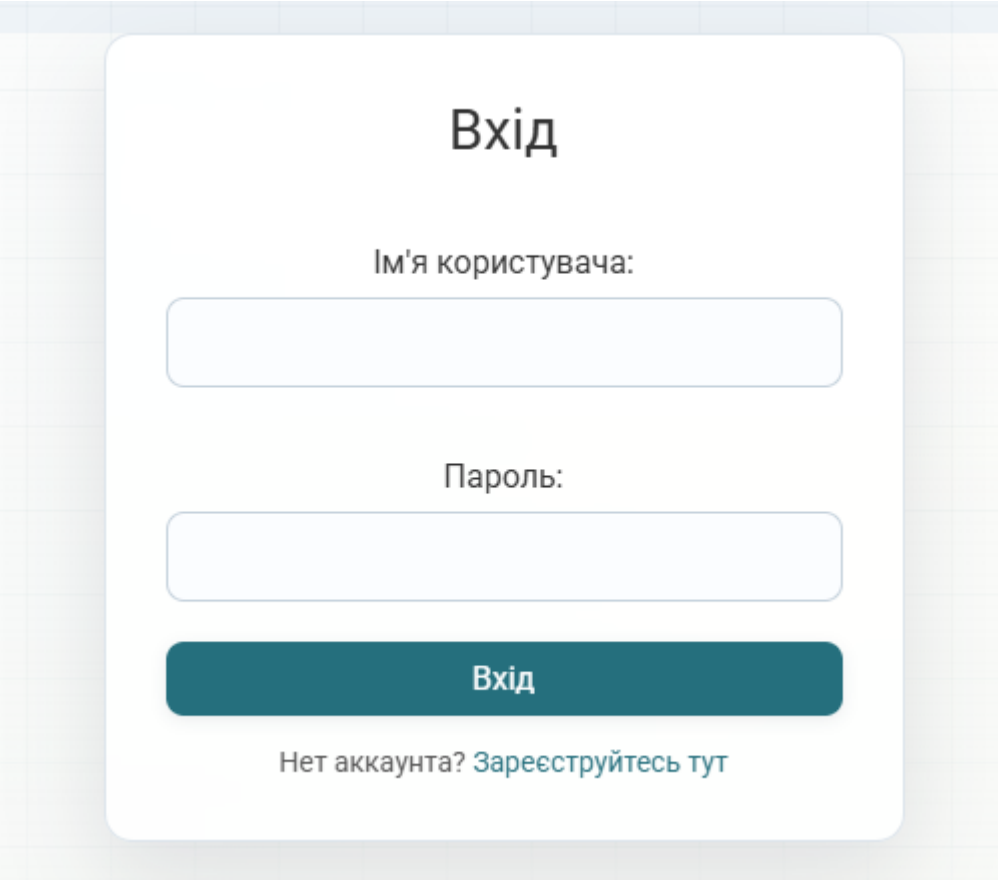


Рисунок 3.3 – Форма логіну

Після невдалої спроби увійти до акаунту можна побачити відповідне повідомлення

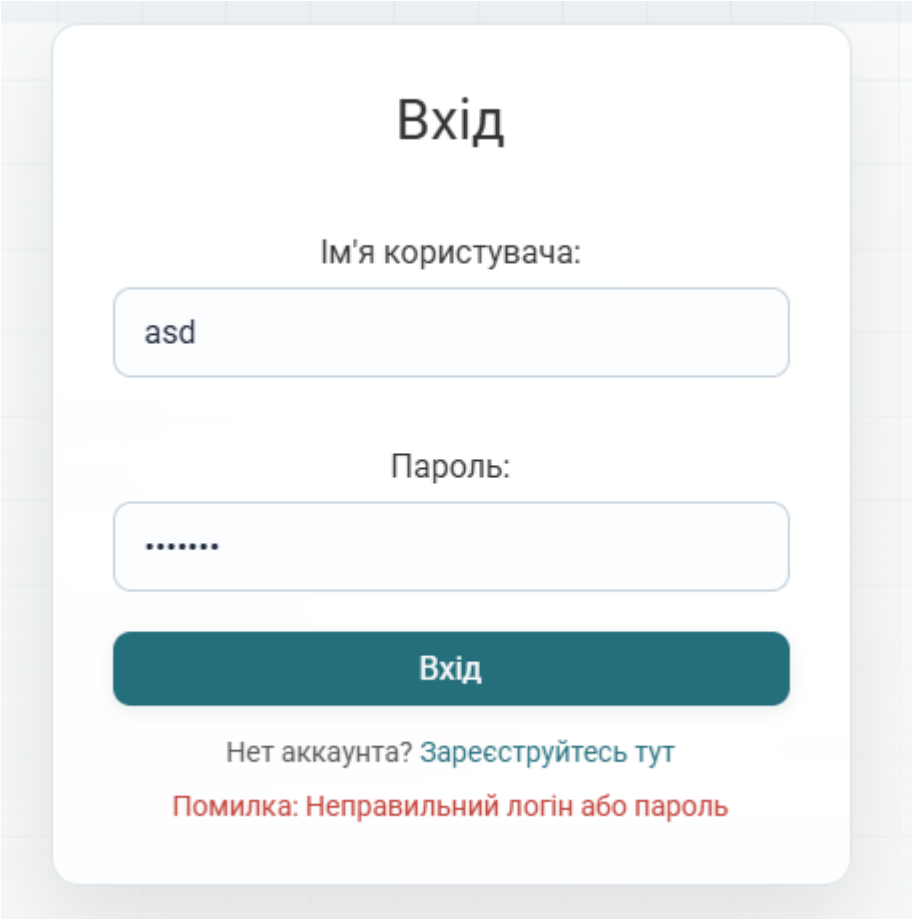


Рисунок 3.4 – Невдала спроба логіну

3.2. Архівування, відновлення та інші функції з сервісу та налаштування

В програмі реалізована функція збереження поточного стану усіх таблиць бази даних, після збереження цей файл відображається на сторінці. По бажанню можна відновити будь-який збережений файл. Цією функцією володіє лише адміністратор

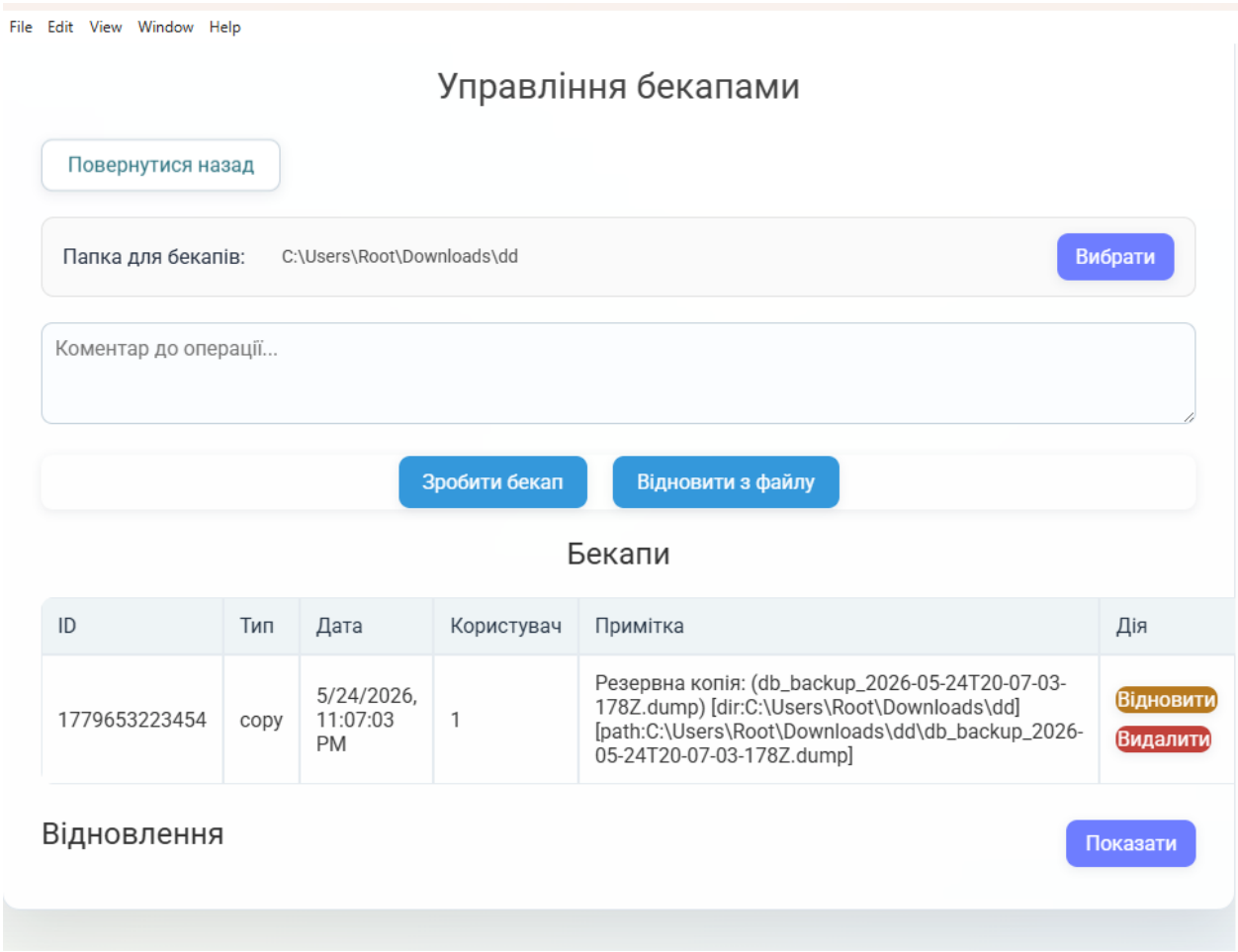


Рисунок 3.5 – Контроль бекапів

При успішному відновленні, адміністратор побаче відповідне повідомлення:

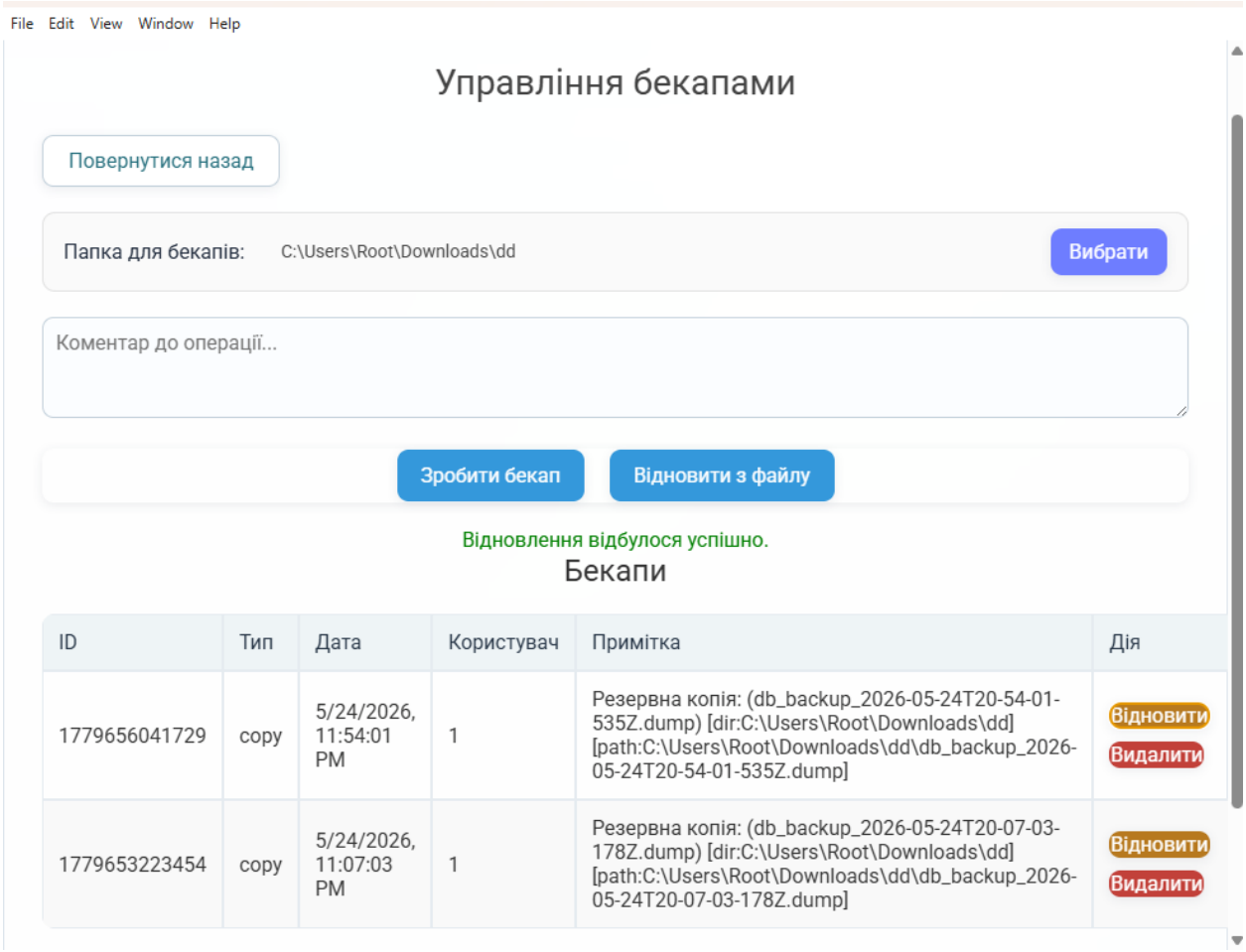


Рисунок 3.6 – Вдала спроба відновити базу даних

3.3. Ведення журналу роботи користувачів системи та його обробка

У застосунку існує можливість відстежувати всі дії користувачів. Кожна подія реєструється, записується тип події, час, хто зробив. Уся ця інформація далі записується до бд. Всю історію можна продивитися на спеціальній сторінці, яка доступна лише адміністратору. Для більш зручного відстежування на сайті є можливіть застосувати фільтри за типом події, за користувачем.

Завдяки цій технології, є можливість слідкувати та відстежувати усі події та бути ознайомленим, що відбувається в програмі.

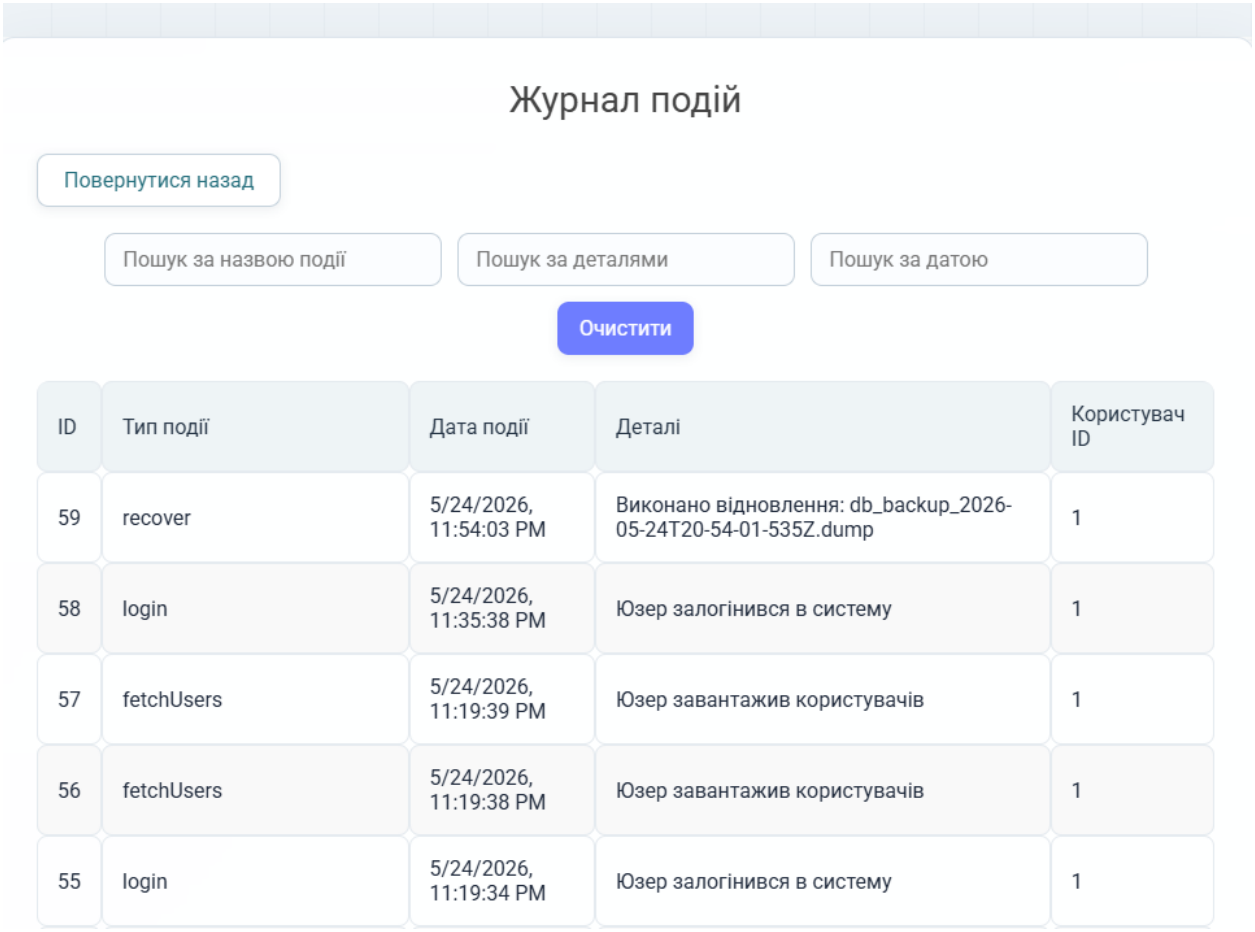


Рисунок 3.7 – Система логування

4 ІНСТРУКЦІЇ ПО ВИКОРИСТАННЮ ТА СУПРОВОДЖЕННЮ КОРИСТУВАЧАМ ЗА РІВНЕМ ДОСТУПУ

Для доступу до застосунку потрібно або зареєструватися, або увійти до акаунту, в залежності від ролі буде доступний різний функціонал. Інтерфейс розроблено таким чином, щоб увесь цей шлях відбувався інтуітивно зрозуміло. Усі форми супроводжуються надписами та ще різною допомогою.

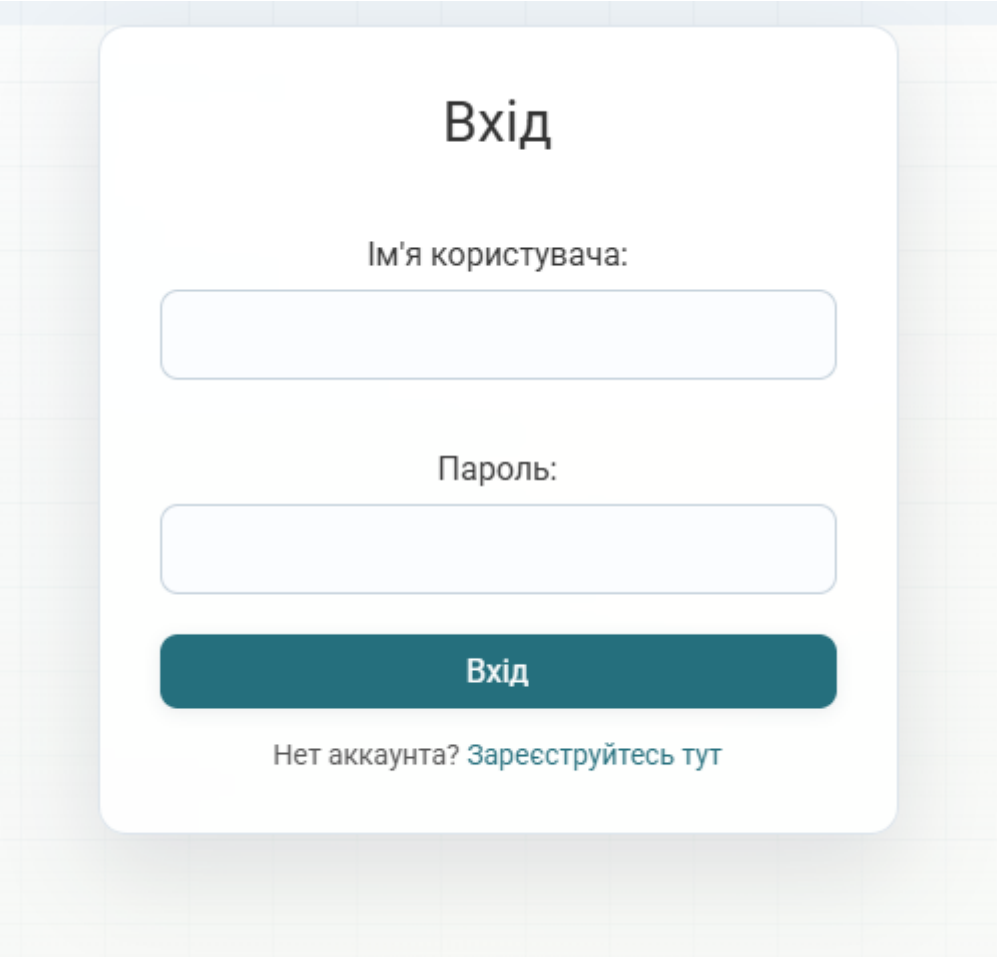


Рисунок 4.1 – Форма логіну

В залежності від ролі, користувач буде мати різний інтерфейс.

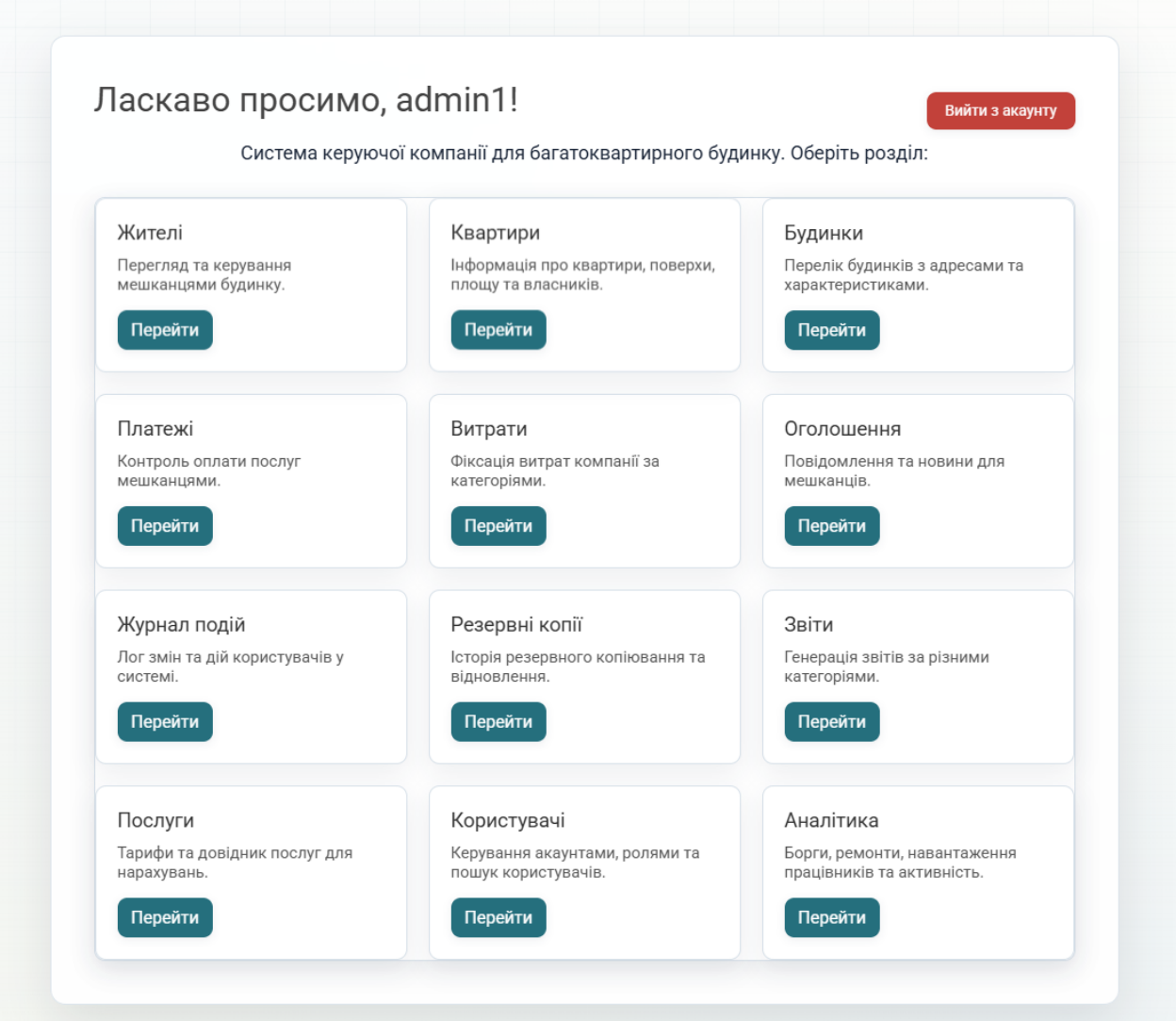


Рисунок 4.2 – Головна сторінка для адміністратора

4.1 Інструкції по використанню та супроводженню адміністратору системи

Інтерфейс було спроєктовано таким, чином, щоб було зрозуміло, куда який елемент UI веде.

У адміністратора доступні наступні функції:

– Жителі. Адміністратор може слідкувати за поточним списком мешканців, робити запити на пошук та видаляти акаунти користувачів.

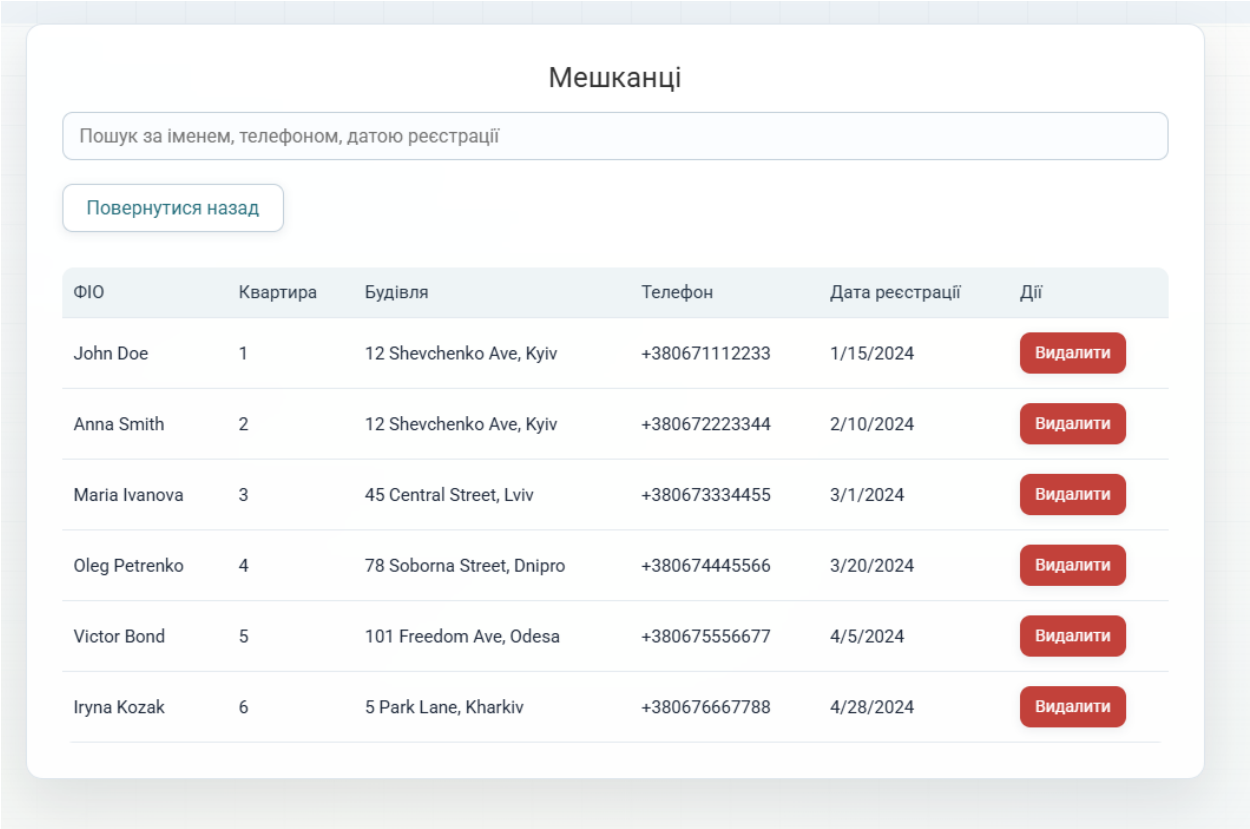


Рисунок 4.3 – Сторінка для моніторингу мешканців

– Квартири. Сторінка, за допомогою якої, можна створювати, редагувати, видаляти, шукати квартири будівель.

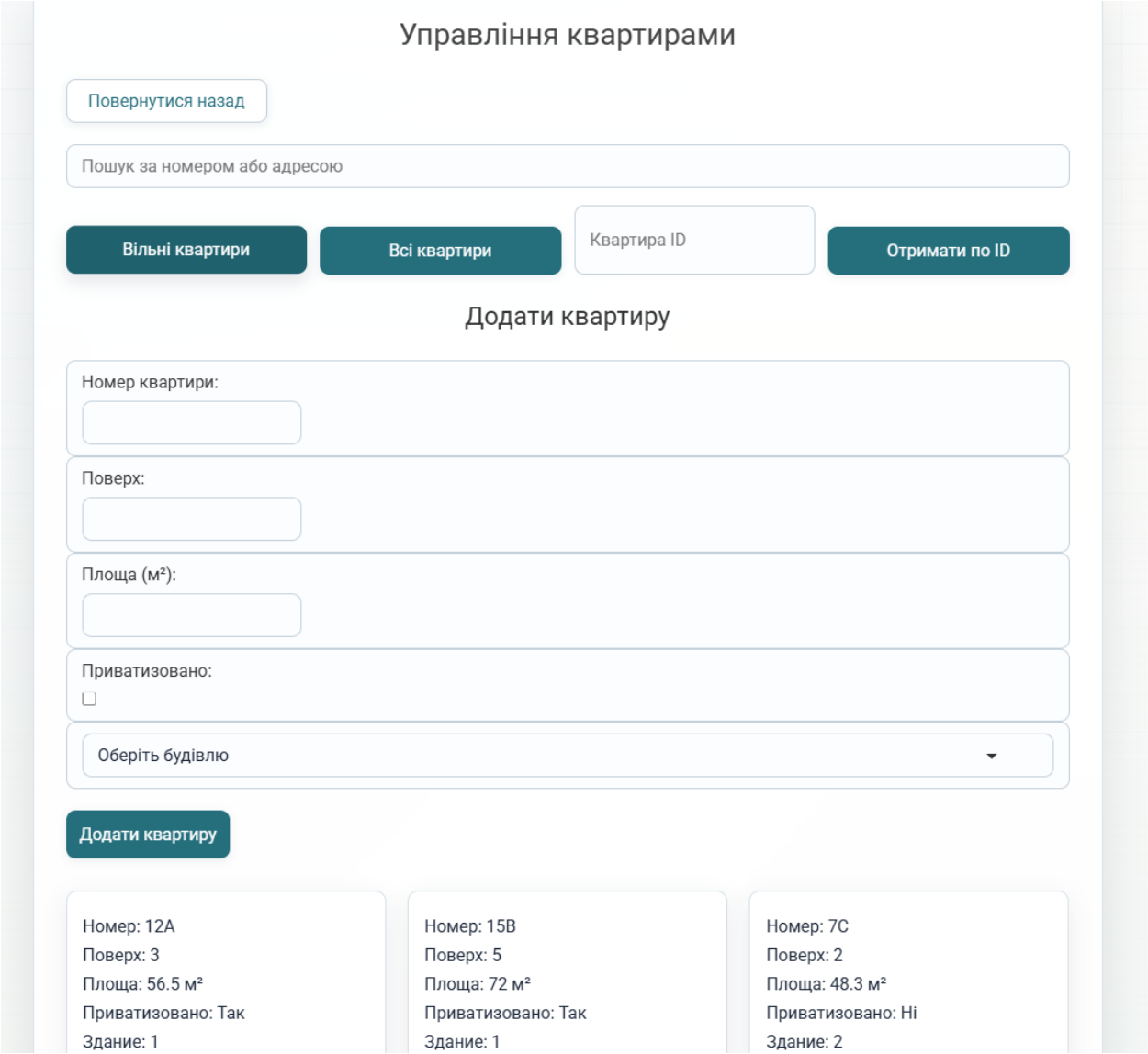


Рисунок 4.4 – Сторінка для керування квартирами

– Будинки. Адміністратор має можливість продивлятися, створювати змінювати інформацію про будинки.

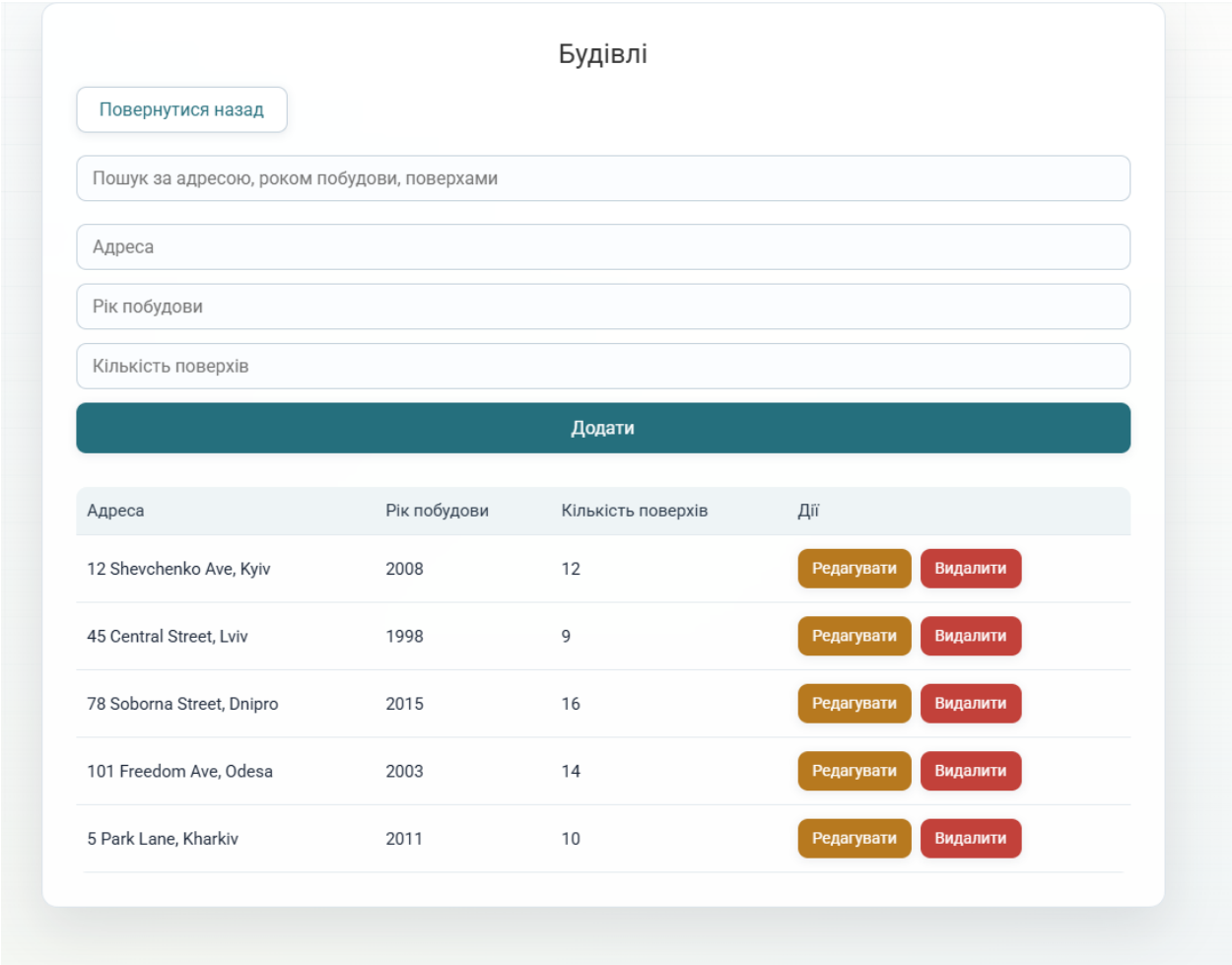


Рисунок 4.5 – Сторінка будівель

– Платежі. На сторінці платежів адміністратор може бачити поточні оплачені та неоплачені платежі усіх мешканців. Може додавати, видаляти, редагувати.

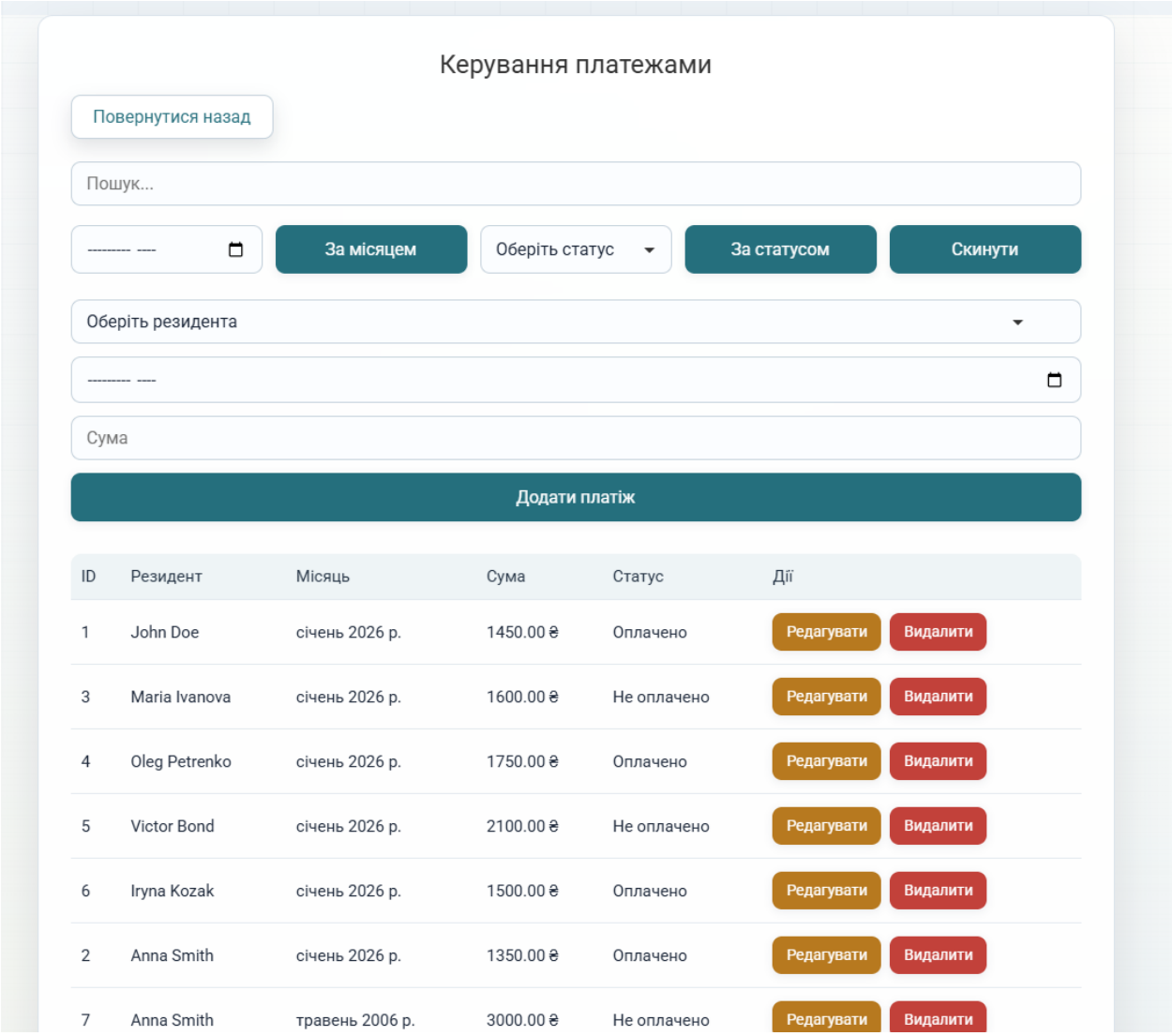


Рисунок 4.6 – Сторінка для управління платежами

– Витрати. Адміністратор Може контролювати усі витрати адміністрації будівель.

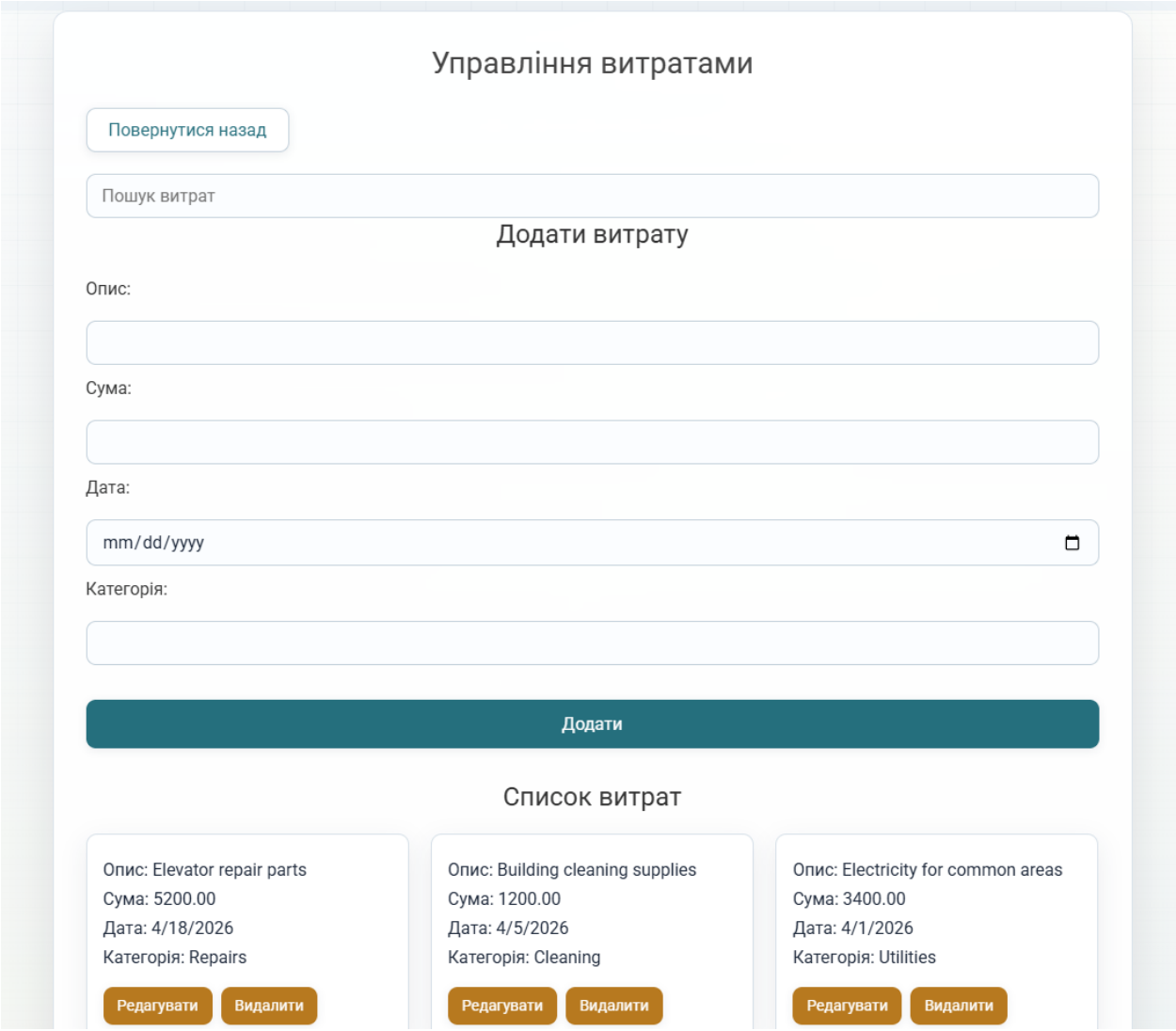


Рисунок 4.7 – Сторінка витрат

– Журнал подій. За допомогою цього розділу, у адміністратора є можливість продивлятися та знати усе, що відбувається у застосунку.

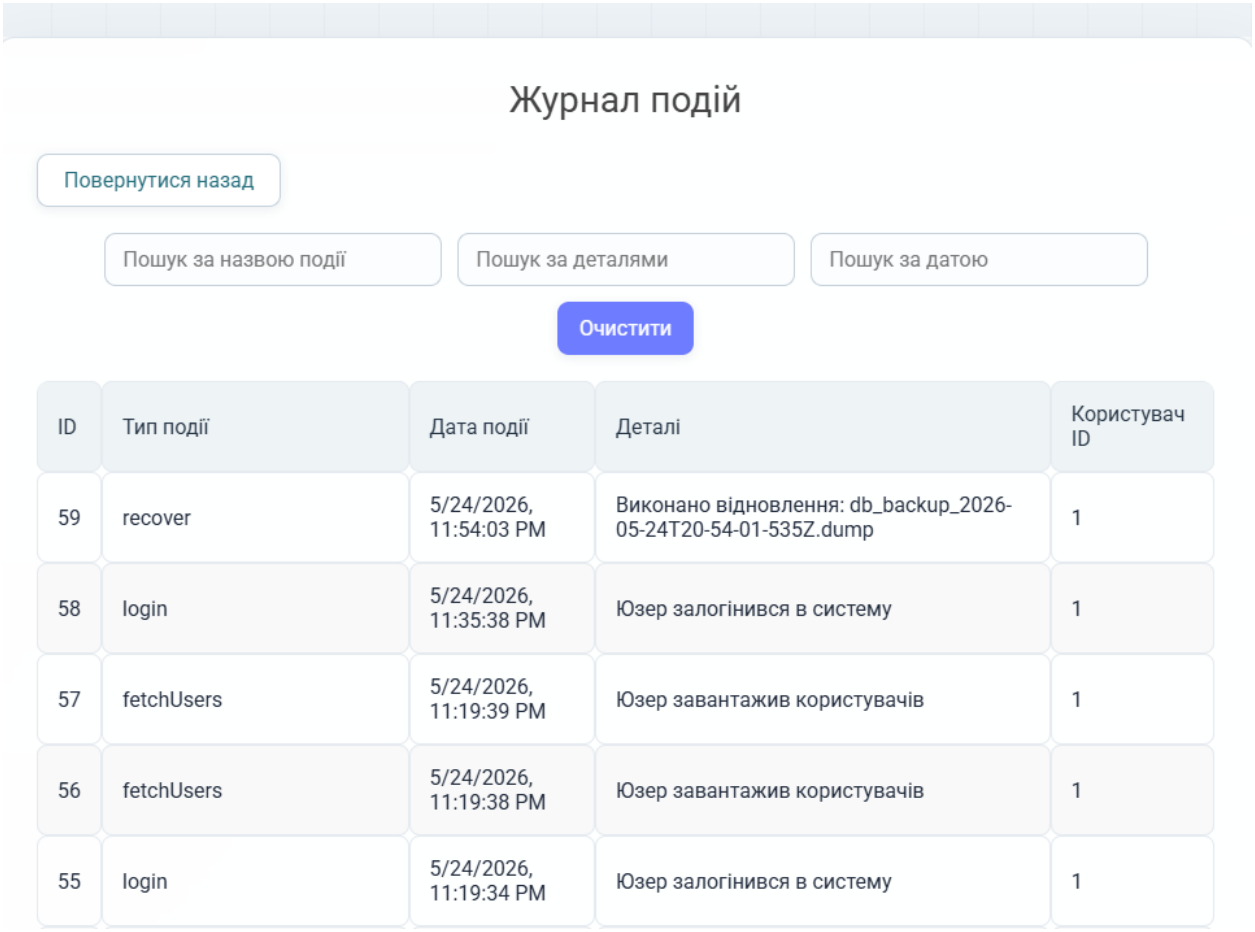


Рисунок 4.8 – Журнал подій

– Резервні копії. Сторінка, функціонал якої, відповідає за створення резервних копій бази даних та відтворення бази даних з файлу.

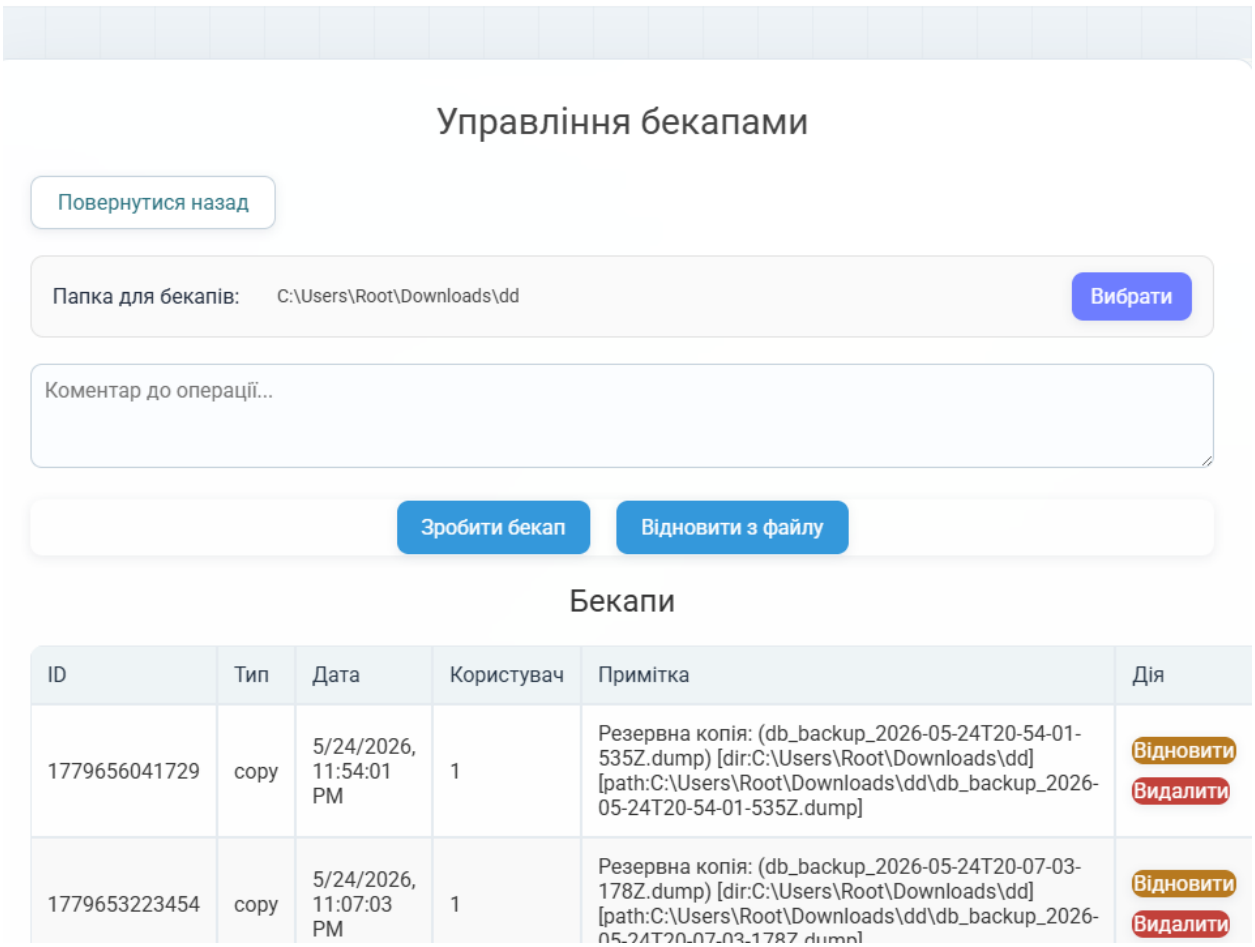


Рисунок 4.9 – Сторінка бекапів

– Звіти. Сторінка, яка дозволяє робити різноманітні запити, отримувати результати у пдф з можливістю роздрукувати та завантажити.

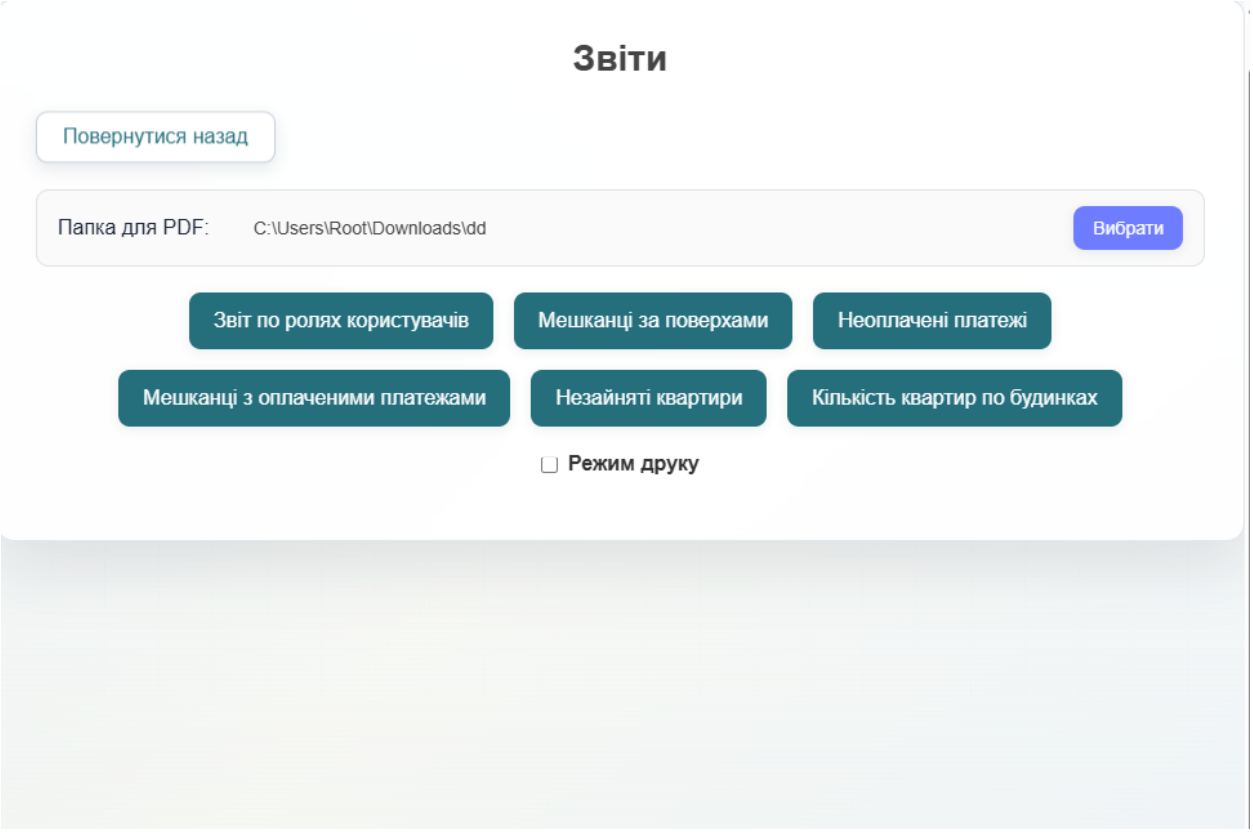


Рисунок 4.10 – Сторінка звітів

4.2 Інструкції по використанню та супроводженню мешканця

Після успішної авторизації чи реєстрації мешканець потрапляє на головну сторінку застосунку, але має менший функціонал.

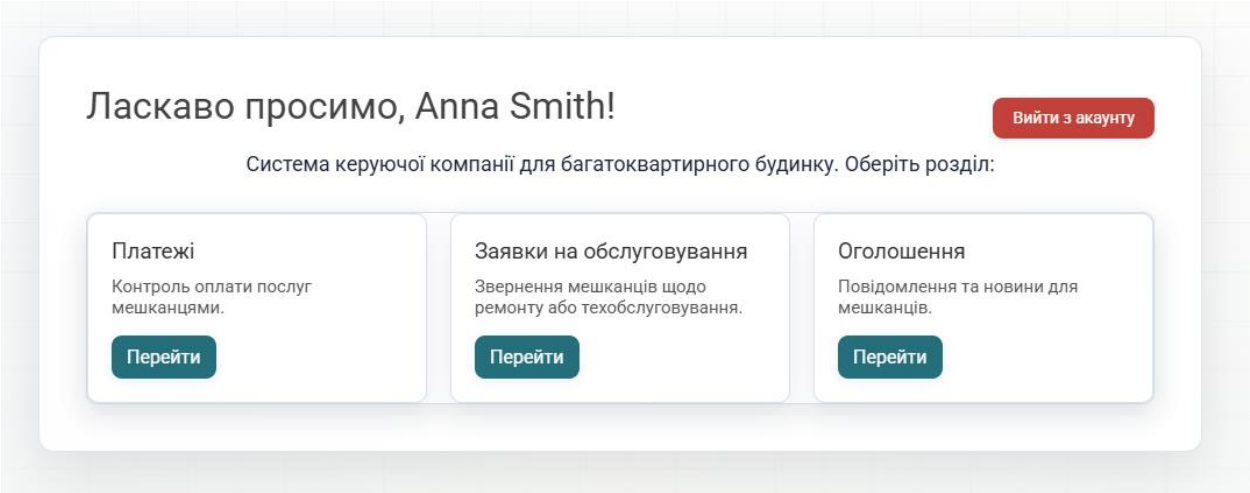


Рисунок 4.11 – Головне меню мешканця

Доступні функції співробітника:

– Платежі. Має можливість передивлятися та оплачувати лише свої платежі

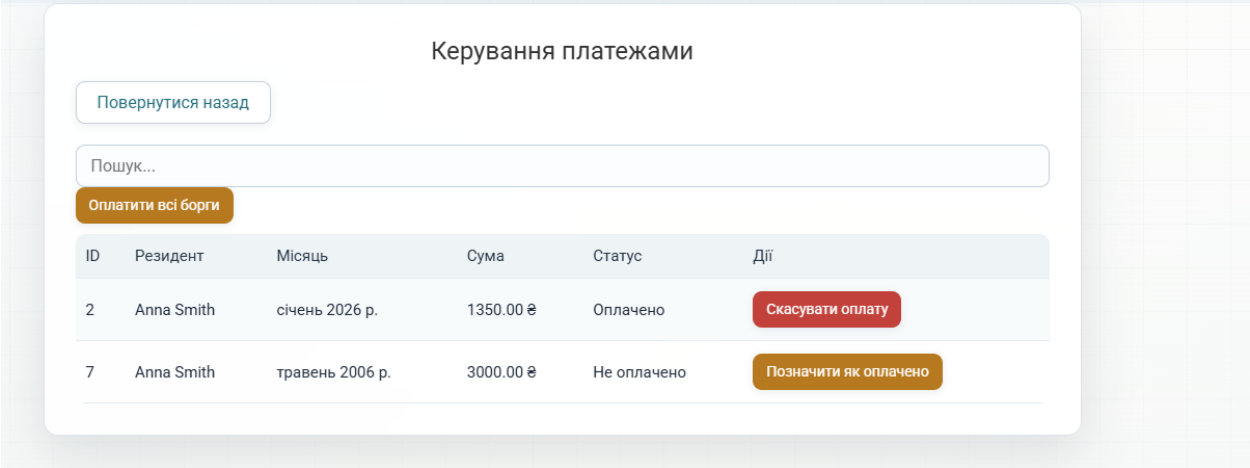


Рисунок 4.12 – Сторінка платежів.

– Заявки на обслуговування. Мешканець може залишати заяву про якусь необхідність та чекати на її виконання.

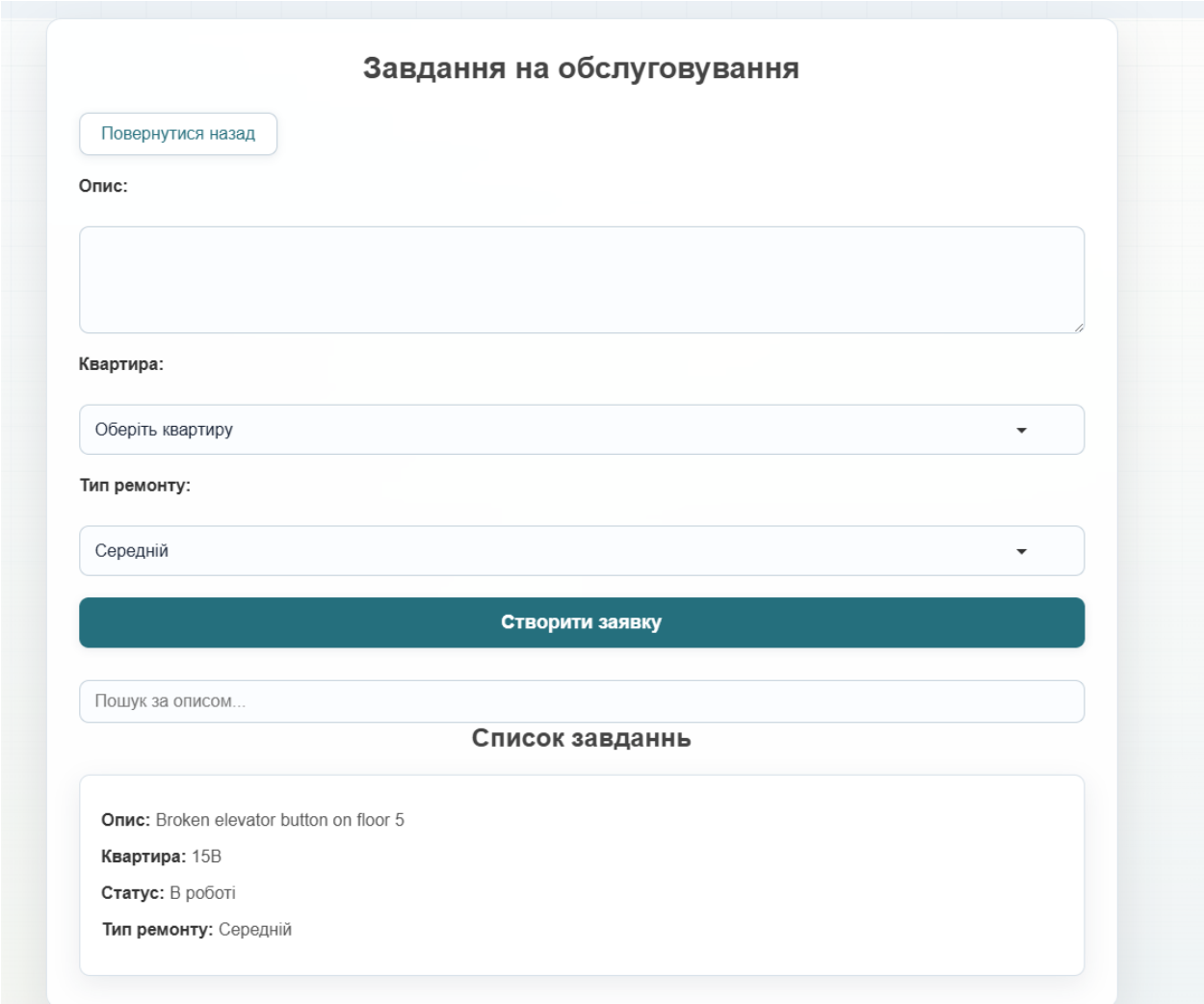


Рисунок 4.13 – Сторінка с заявами на обслуговування.

– Оголошення. Ця сторінка дозволяє залишати оголошення для всіх мешканців, також є можливість редагувати, видаляти ці оголошення.

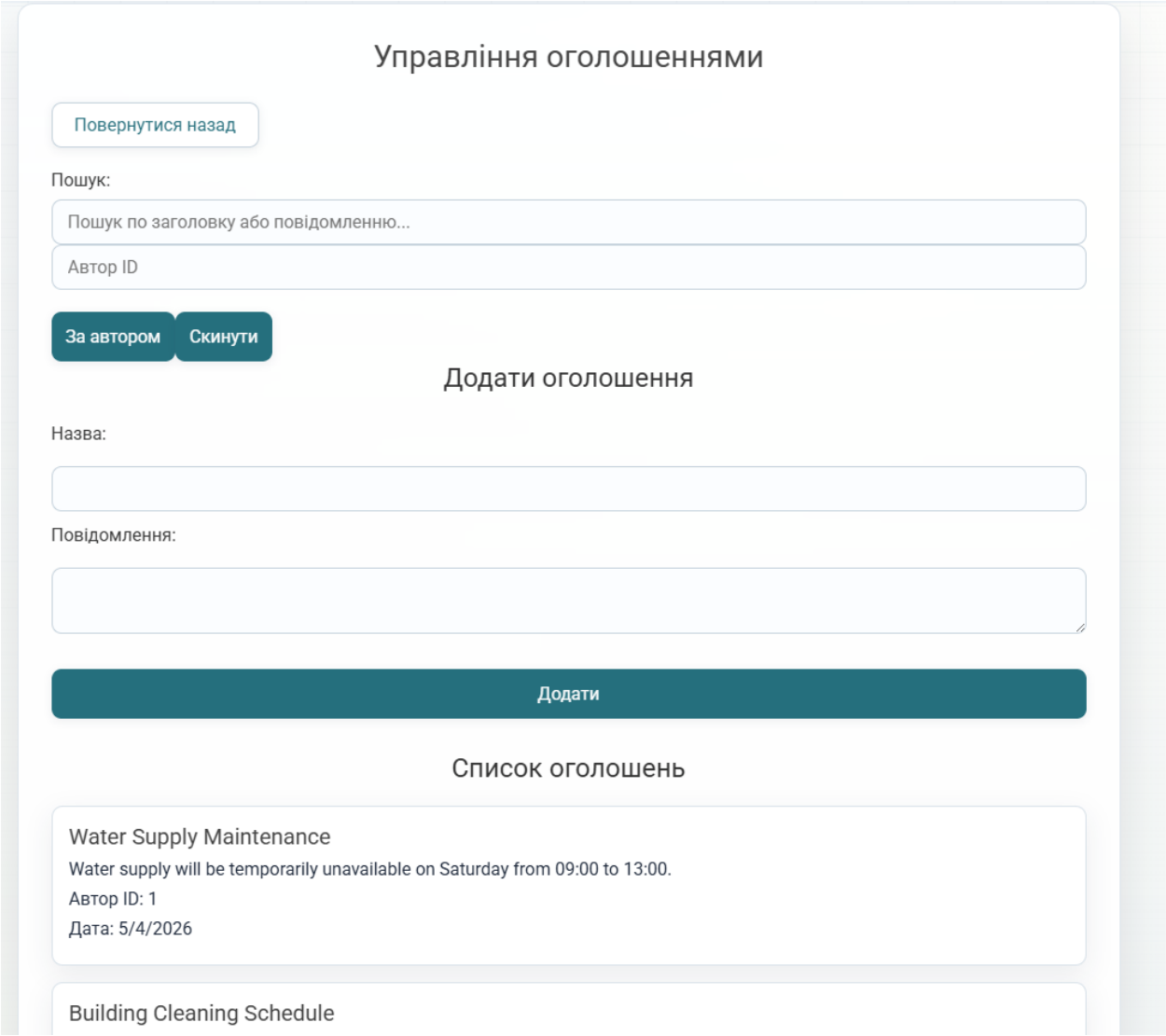


Рисунок 4.14 – Оголошення

4.3 Інструкції по використанню та супроводженню працівника

При авторизації, як працівник, знову з’являється те ж саме меню, але з різним функціоналом.

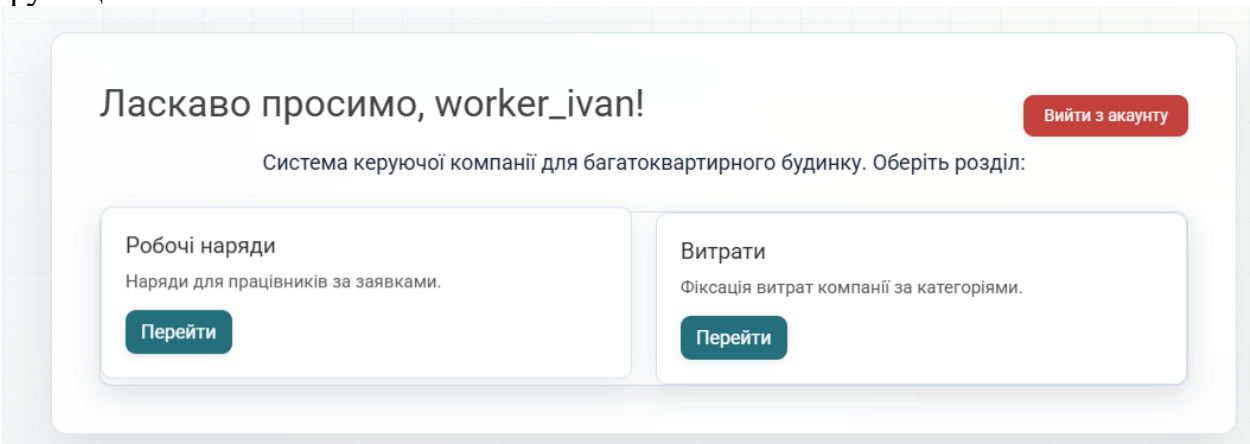


Рисунок 4.15 – Основний інтерфейс працівника

– Робочі наряди. Головний розділ для працівника, саме тут він буде брати собі роботу.

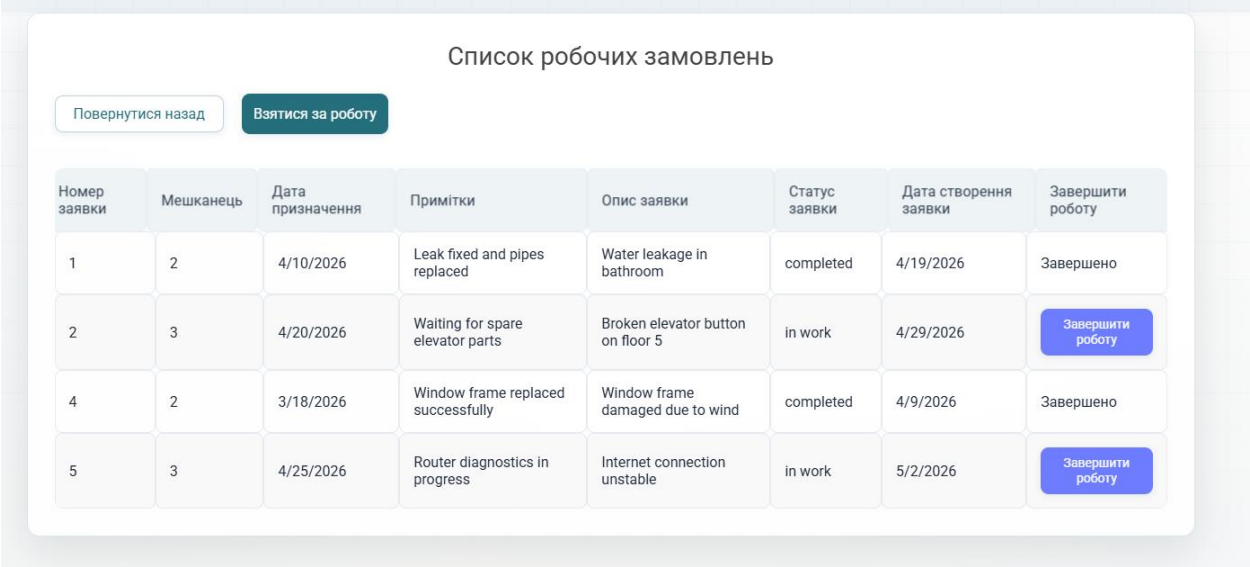


Рисунок 4.16 – Список робочих замовлень.

– Витрати. Сторінка для відстежування витрат компанії.

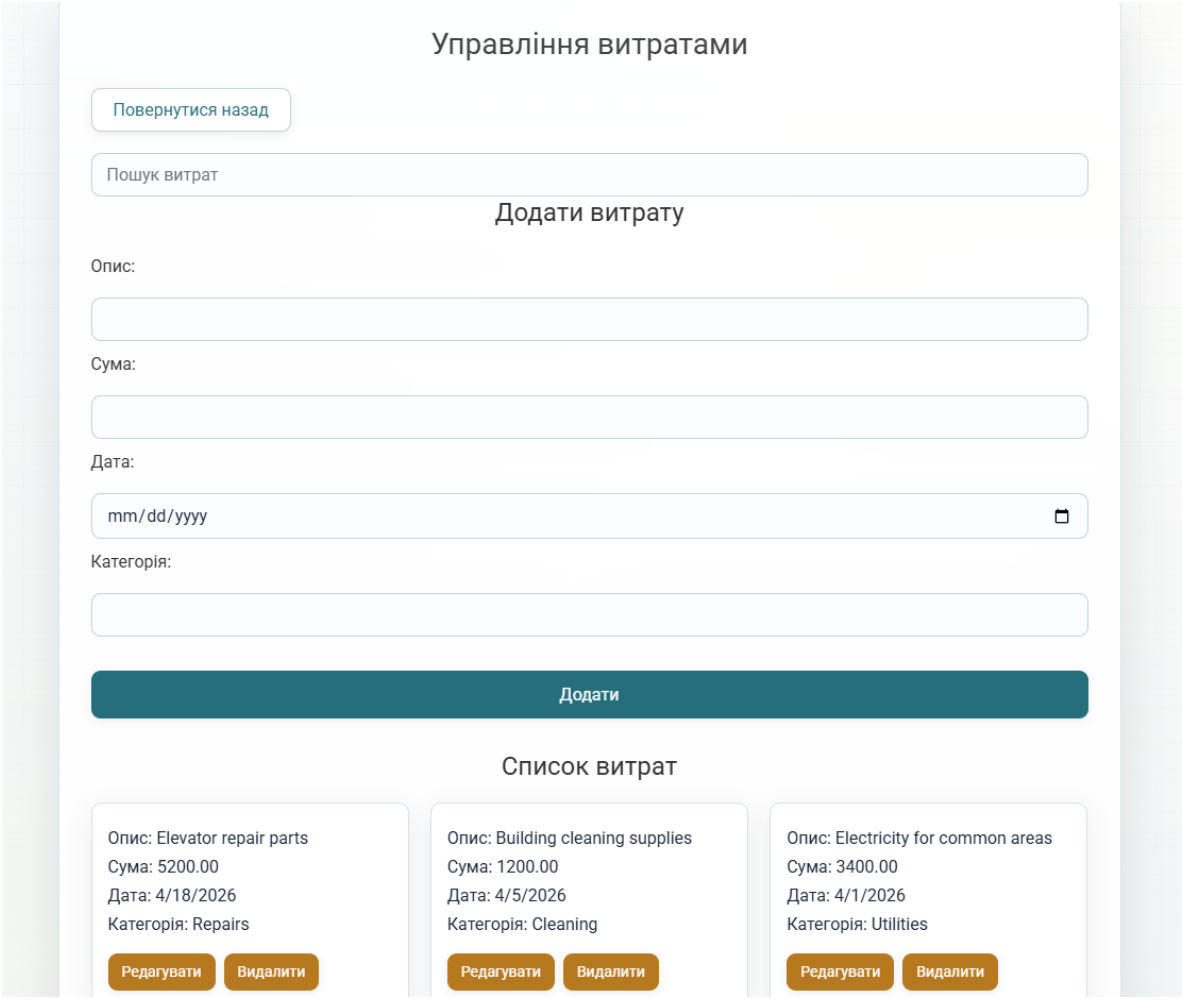


Рисунок 4.17 – окно з витратами

Отже, для кожної ролі були зроблені свої обмеження, Усе зроблено зрозуміло, легке для взаємодії.

5 АНАЛІЗ ТА ОБГРУНТУВАННЯ РЕЗУЛЬТАТІВ ТЕСТУВАННЯ ТА НАЛАГОДЖЕННЯ

5.1. Початкове введення даних

Щоб забезпечити коректний ввід даних в формі, на клієнті відбувається валідація даних перед відправкою до серверу, якщо є якісь негаразди, то користувач миттєво отримує відповідне повідомлення. До таких перевірок відноситься перевірка правильно уведеного паролю та періврка коректного уведення пошти.

5.2. Базове функціонування

Для повноцінного тестування застосунку, було прийнято рішення пройти такий алгоритм: створення будинку, квартири, резервної копії, платежа, відновлення резервної копії.

1. Спочатку адміністратор переходить до розділу з будівлями, вводить адресу, рік побудови, кількість поверхів у відповідні поля, натискає на кнопку “Додати”, в результаті будівля створена.
2. Адміністратор повертається до головної сторінки, переходить до розділу квартир, вводить номер квартири, поверх, площу, обирає доступну будівлю та натискає кнопку “Додати квартиру”.
3. Адмін повертається до головного меню, переходить до розділу резервних копій, натискає на кнопку “Зробити бекап”, очікує на повідомлення.
4. Для здійснення платежу, адмін обирає відповідний розділ, обирає мешканця, пише суму та вибрає дату, в кінці натиснувши “Додати платіж”.
5. В кінці, щоб перевірити функціональність відновлення резервної копії, адміністратор переходить до відповідного розділу, натискає на кнопку “Відновити” та очікує на повідомлення.

Отже, після всіх цих дій, можна переконатися, що увесь функціонал в робочому стані, працює не лише CRUD, але й створення та відтворення резервної копії.

5.3. Тестування підсистеми безпеки та логування

1. Автентифікація: Тестування підтвердило неможливість входу в систему з некоректними даними. Паролі успішно трансформуються у незворотні хеші за алгоритмом SHA-256.

2. Контроль доступу: Процедура відпрацьовує без збоїв. Наприклад, при авторизації під роллю «Мешканець», програма миттєво адаптує інтерфейс, прибираючи усі дії, що не може роботи мешканець.

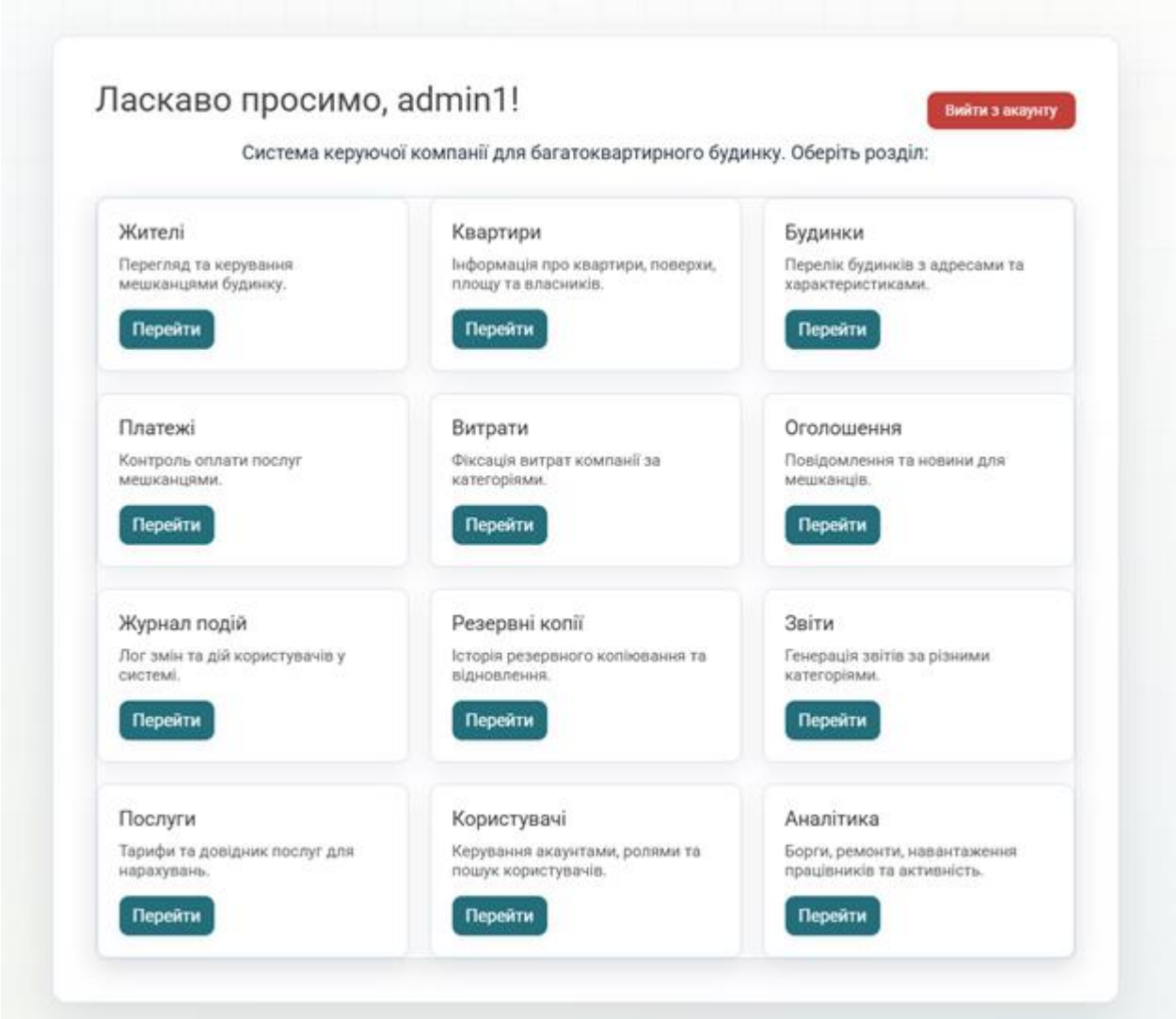


Рисунок 5.1 – Вхід під адміністратором

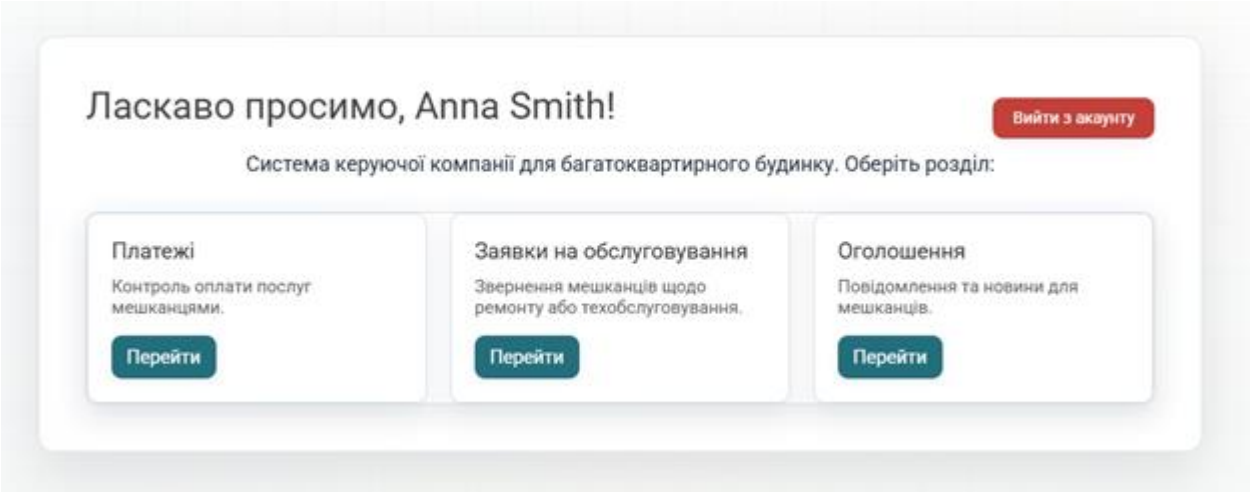


Рисунок 5.2 – Вхід під мешканцем

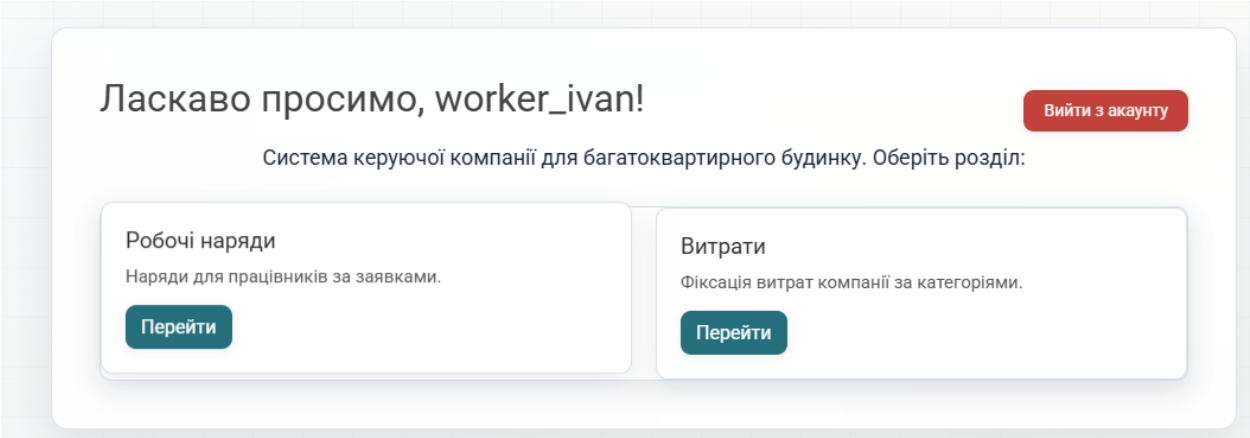


Рисунок 5.3 – Вхід під робітником

3. Журнал подій: Перевірка таблиці logs показала, що кожна критична дія (від входу в систему до видалення будинку чи генерації звіту) автоматично фіксується з точним зазначенням часу та автором.

5.4. Максимальне навантаження

Було створено код, який генерує несправжні дані таблиць та додає їх до бази даних.

Код генерації даних:

```
import { faker } from "@faker-js/faker";
import pkg from "pg";
import dotenv from "dotenv";

dotenv.config();

const { Pool } = pkg;

const pool = new Pool({
  user: "postgres",
  host: "localhost",
  database: "course_project",
  password: "1",
  port: 5432,
});

const USER_COUNT = 500;
```

```
const BUILDING_COUNT = 20;
const APARTMENT_COUNT = 200;
const RESIDENT_COUNT = 150;
const REQUEST_COUNT = 300;
const PAYMENT_COUNT = 500;
const WORK_ORDER_COUNT = 200;

async function time(label, fn) {
  console.time(label);
  try {
    return await fn();
  } finally {
    console.timeEnd(label);
  }
}

const random = (arr) => arr[Math.floor(Math.random() * arr.length)];

async function batchInsert(client, query, values) {
  for (const v of values) {
    await client.query(query, v);
  }
}

async function seedUsers() {
  const client = await pool.connect();
  const roles = ["resident", "admin", "worker"];

  const users = [];

  for (let i = 0; i < USER_COUNT; i++) {
    users.push([
      faker.internet.username().toLowerCase() + i,
      "12345678",
      faker.internet.email(),
```

```
        random(roles),
      ]);
    }

    await client.query("BEGIN");

    try {
      await batchInsert(
        client,
        `INSERT INTO users (username, password_hash, email,
role)
         VALUES ($1, $2, $3, $4)`,
        users
      );

      await client.query("COMMIT");
    } catch (e) {
      await client.query("ROLLBACK");
      throw e;
    } finally {
      client.release();
    }
  }

  async function seedBuildings() {
    const client = await pool.connect();

    const buildings = [];

    for (let i = 0; i < BUILDING_COUNT; i++) {
      buildings.push([
        faker.location.streetAddress(),
        faker.number.int({ min: 1960, max: 2024 }),
        faker.number.int({ min: 3, max: 20 }),
      ]);
    }
```

```
    await client.query("BEGIN");

    try {
      await batchInsert(
        client,
        `INSERT INTO buildings (address, built_year,
number_of_floors)
         VALUES ($1, $2, $3)`,
        buildings
      );

      await client.query("COMMIT");
    } finally {
      client.release();
    }
  }

  async function seedApartments() {
    const client = await pool.connect();

    const buildings = await client.query("SELECT id FROM
buildings");
    const buildingIds = buildings.rows.map((r) => r.id);

    const apartments = [];

    for (let i = 0; i < APARTMENT_COUNT; i++) {
      apartments.push([
        random(buildingIds),
        `${faker.number.int({ min: 1, max: 999 })}`,
        faker.number.int({ min: 1, max: 20 }),
        faker.number.float({ min: 20, max: 120 }),
        faker.datatype.boolean(),
        null,
      ]);
```

```
    }

    await client.query("BEGIN");

    try {
      await batchInsert(
        client,
        `INSERT INTO apartments (building_id, number, floor,
area, owned, user_id)
         VALUES ($1, $2, $3, $4, $5, $6)`,
        apartments
      );

      await client.query("COMMIT");
    } finally {
      client.release();
    }
  }

  async function seedResidents() {
    const client = await pool.connect();

    const users = await client.query(
      "SELECT id FROM users WHERE role = 'resident'"
    );
    const apartments = await client.query("SELECT id,
building_id FROM apartments");

    const residents = [];

    for (let i = 0; i < RESIDENT_COUNT; i++) {
      const apartment = random(apartments.rows);

      residents.push([
        random(users.rows).id,
        faker.person.fullName(),
```

```
            apartment.id,
            apartment.building_id,
            faker.phone.number(),
            faker.date.past(),
          ]);
        }

        await client.query("BEGIN");

        try {
          await batchInsert(
            client,
            `INSERT INTO residents (user_id, full_name,
apartment_id, building_id, phone, registered_at)
             VALUES ($1, $2, $3, $4, $5, $6)`,
            residents
          );

          await client.query("COMMIT");
        } finally {
          client.release();
        }
      }

      async function seedRequests() {
        const client = await pool.connect();

        const residents = await client.query("SELECT id FROM
residents");
        const apartments = await client.query("SELECT id FROM
apartments");

        const types = ["cosmetic", "medium", "major", "emergency"];
        const status = ["new", "in work", "completed"];

        const requests = [];
```

```
    for (let i = 0; i < REQUEST_COUNT; i++) {
      requests.push([
        random(residents.rows).id,
        random(apartments.rows).id,
        faker.lorem.sentence(),
        random(types),
        random(status),
        faker.date.recent(),
        faker.date.recent(),
      ]);
    }

    await client.query("BEGIN");

    try {
      await batchInsert(
        client,
        `INSERT INTO maintenance_requests
         (resident_id, apartment_id, description, repair_type,
status, created_at, updated_at)
         VALUES ($1,$2,$3,$4,$5,$6,$7)`,
        requests
      );

      await client.query("COMMIT");
    } finally {
      client.release();
    }
  }

  async function seedPayments() {
    const client = await pool.connect();

    const residents = await client.query("SELECT id FROM
residents");
```

```
    const status = ["payed", "not payed"];

    const payments = [];

    for (let i = 0; i < PAYMENT_COUNT; i++) {
      payments.push([
        random(residents.rows).id,

        // FIX: proper DATE format
        faker.date.between({
          from: "2023-01-01",
          to: "2026-01-01",
        }),

        faker.number.float({ min: 100, max: 2000 }),
        faker.date.recent(),
        random(status),
      ]);
    }

    await client.query("BEGIN");

    try {
      await batchInsert(
        client,
        `INSERT INTO payments (resident_id, month, amount,
paid_at, status)
         VALUES ($1,$2,$3,$4,$5)`,
        payments
      );

      await client.query("COMMIT");
    } finally {
      client.release();
    }
  }
```

```
async function seedWorkOrders() {
  const client = await pool.connect();

  const requests = await client.query("SELECT id FROM
maintenance_requests");
  const workers = await client.query(
    "SELECT id FROM users WHERE role = 'worker'"
  );

  const workOrders = [];

  for (let i = 0; i < WORK_ORDER_COUNT; i++) {
    workOrders.push([
      random(requests.rows).id,
      random(workers.rows)?.id || null,
      faker.date.future(),
      faker.date.recent(),
      faker.lorem.sentence(),
    ]);
  }

  await client.query("BEGIN");

  try {
    await batchInsert(
      client,
      `INSERT INTO work_orders (request_id, worker_id,
scheduled_date, completed_at, notes)
       VALUES ($1,$2,$3,$4,$5)`,
      workOrders
    );

    await client.query("COMMIT");
  } finally {
    client.release();
```

```
  }
}

// -------------------- RUN ALL --------------------
async function run() {
  console.time("TOTAL SEED TIME");

  await time("users", seedUsers);
  await time("buildings", seedBuildings);
  await time("apartments", seedApartments);
  await time("residents", seedResidents);
  await time("requests", seedRequests);
  await time("payments", seedPayments);
  await time("work_orders", seedWorkOrders);

  console.timeEnd("TOTAL SEED TIME");

  await pool.end();
}

run().catch(console.error);
    console.timeEnd("users_seed");
```

```
PS E:\Desktop\PracticWork\University\БД\CourseProject\course-work\server\tests> node fakes.js
users: 128.178ms
buildings: 3.667ms
apartments: 28.501ms
residents: 34.602ms
requests: 59.929ms
payments: 72.914ms
work_orders: 31.13ms
TOTAL SEED TIME: 360.849ms
```

Рисунок 5.1 – Результат стрес тестів

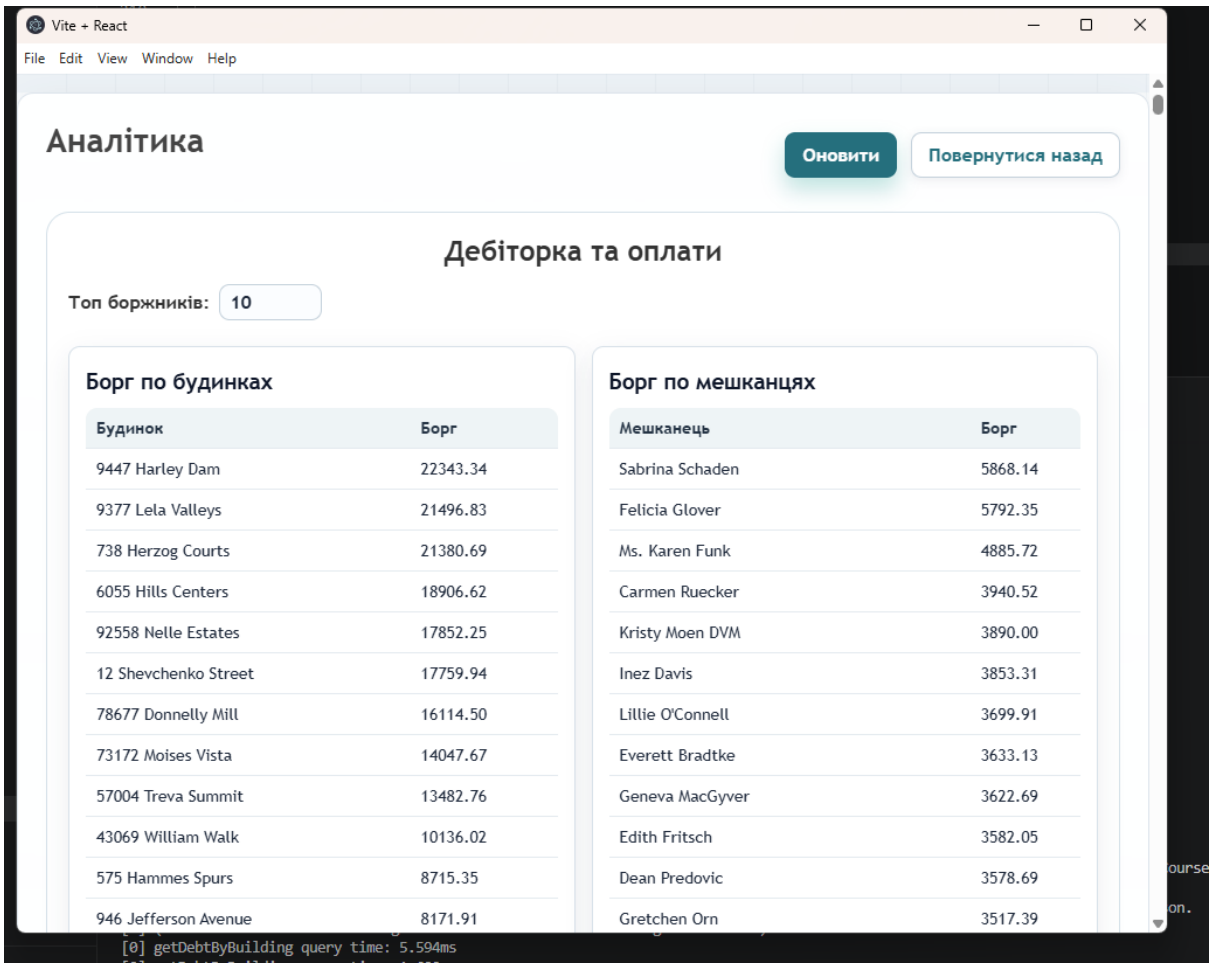


Рисунок 5.2 – Результат стрес тестів

За результатом стрес тесту можна побачити, що запит виконується за 6 мс, що демонструє чудову продуктивність не дивлячись на великий об’єм опрацьованих даних. Алгоритми працюють коректно, база даних захищена від аномалій та перевантажень, а інтерфейс стійкий до непередбачених дій користувачів.

ВИСНОВКИ

У результаті виконання курсової роботи було створено повнофункціональну веб-систему для автоматизації роботи житлового комплексу. Система охоплює ключові аспекти управління мешканцями, квартирами, будівлями, платежами, технічним обслуговуванням та адміністративними процесами. Бекенд реалізовано на Node.js (Express) з використанням PostgreSQL для зберігання даних. Взаємодія з БД побудована на чистих SQL-запитах через pg без ORM, що забезпечує гнучкість і повний контроль над логікою запитів. Фронтенд написано на React, з використанням React Router для клієнтської маршрутизації.

Під час проєктування було здійснено повне обстеження предметної області, ідентифіковано основні сутності – клієнтів, кредити, платежі, співробітників, графіки платежів та дії користувачів. Усі дані організовано у структуровану реляційну модель із багаторівневим контролем доступу. Для кожного типу користувача – менеджера, адміністратора та супер-адміністратора – реалізовано унікальні сценарії доступу відповідно до рівня відповідальності.

У системі реалізовано:

- CRUD-операції з усіма таблицями, користувачами та їх ролями (resident, admin, worker);
- Прив’язка мешканців до квартир і будівель;
- Пошук і фільтрація за ПІБ, адресою або номером квартири;
- Облік та статистика платежів;
- Реєстрація щомісячних платежів мешканців;
- Автоматичне оновлення статусу платежів (payed, not payed).
- Усі важливі дії (реєстрації, зміни, видалення) фіксуються у журналі подій
- Генерація PDF-звітів
- Підтримка створення та відновлення резервних копій бази даних через інтерфейс

Система протестована під навантаженням з десятками тисяч одночасних звернень, і показала стабільну роботу. Завдяки модульній архітектурі легко масштабувати функціональність, додавати нові модулі.

Проєкт реалізовано на сучасному стеку React + Node.js + PostgreSQL, з мінімалістичним підходом (без ORM) і чіткою архітектурою. Забезпечено високий рівень безпеки, масштабованість і гнучкість, що дозволяє впровадити систему в реальних умовах житлового комплексу або керуючої компанії.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

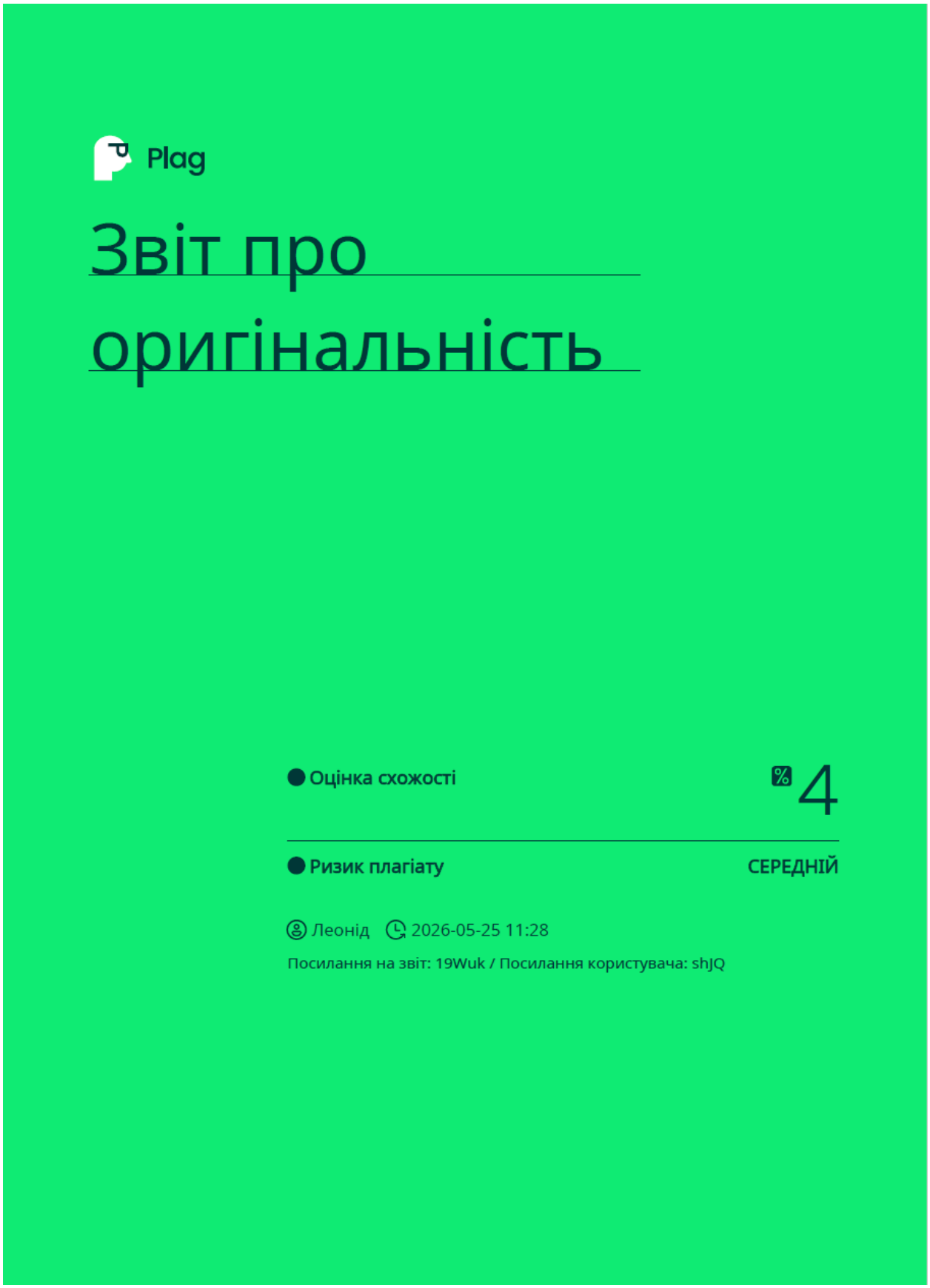
1. Node js. URL: https://nodejs.org/uk (Дата звернення: 1.05.2025)
2. postgreSQL. URL: https://www.postgresql.org/ (Дата звернення: 25.04.2025)
3. React. URL: https://react.dev/ (Дата звернення 1.05.2025)
4. Metanit. URL: https://metanit.com/web/nodejs/ (Дата звернення 1.05.2025)
5. Документація до бібліотеки для роботи з PDF – jsPDF. URL: https://www.npmjs.com/package/jspdf (Дата звернення 2.05.2025)
6. Мащенко Л.В. Методичні вказівки до самостійної роботи з дисципліни «Бази даних»: Дніпро, ДНУ імені Олеся Гончара, 2018. – 29 с. URL: http://repository.dnu.dp.ua:1100/?page=inner_material&id=9828 (Дата звернення 1.05.2025)

ДОДАТОК 1

Дані користувацьких акаунтів для тесутвання

1) Адмін:
   Логін : admin1
   Пароль: pswd
2) Мешканець:
   Логін: anna_smith
   Пароль: qwe
3) Працівник:
   Логін: worker_ivan
   Пароль: 1234

ДОДАТОК 2
Звіт про оригінальність





Ось вона –
Ваша звіт про оригінальність!

Ми раді повідомити, що перевірка вашого документа завершена, і результати вже готові! Наші алгоритми старанно працювали, щоб знайти збіги в наших базах даних.

На наступних сторінках ви знайдете результати перевірки:

Бали

Збіги

Посилання

Ваш документ було перевірено за такими джерелами:

- [x] База даних інтернет-джерел
- [ ] База даних наукових статей
- [ ] Глибока перевірка (наш вдосконалений алгоритм)

Бали



Збіги тексту	4%
Перефразування	0%
Цитований текст	0%
Неправильне цитування	0%
Збігів не знайдено	96%

Ризик плагіату СЕРЕДНІЙ

Ризик плагіату вказує, як збіги тексту розподілені по документу. Вищий ризик виникає, коли збіги з'являються близько один до одного, наприклад, у тому самому абзаці або розділі.

Оцінка схожості 4

Оцінка схожості показує, скільки слів або символів у вашому документі збігаються з текстами інших документів, включаючи перефразовані тексти або неправильні цитати.